PROGRAMACIÓN CON ARDUINO · CLASE 3

Sensores: el Arduino aprende a escuchar

Potenciómetro, pulsador, buzzer, display y cinco sensores del kit. Leemos el mundo, lo vemos en el Monitor Serie y reaccionamos.

Entradas · analogRead() · Monitor Serie · Retos



PUNTO DE PARTIDA

De hablar a escuchar: entradas y salidas

Las clases 1 y 2 fueron de **salidas**: el Arduino hablaba. Hoy trabajamos las **entradas**: el Arduino escucha.

1. ENTRADA

Un **sensor** mide algo del mundo: luz, sonido, humedad, si presionaste un botón…

`digitalRead()` `analogRead()`

2. PROCESO

El Arduino **decide** qué hacer con esa información según tu programa.

`if` `map()` `Serial`

3. SALIDA

Algo **reacciona**: un LED se enciende, el buzzer suena, el display muestra un número.

`digitalWrite()` `analogWrite()`

Entrada digital: ¿SÍ o NO?

Solo dos respuestas: **HIGH** o **LOW**. Se lee con `digitalRead()`.
Ej.: pulsador, sensor IR.

Entrada analógica: ¿CUÁNTO?

Un número de **0 a 1023**. Se lee con `analogRead()` en A0–A5.
Ej.: potenciómetro, humedad, nivel de agua.

Ejemplo: el sensor de humedad mide la tierra → el Arduino ve que está seca → el buzzer avisa “¡riégame!”.

TU KIT

Lo que trae el kit de cada pareja



Arduino UNO

El cerebro



Protoboard

Conectar sin soldar



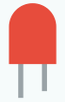
Jumpers

Los cables



Resistencias

Limitan la corriente



LEDs

Salida



Pulsadores

Entrada digital



Potenciómetro

Entrada analógica



Buzzer TMB12A05

Salida (sonido)



Display 7 segmentos

Salida (números)



Sensor IR

Entrada digital



Sensor de sonido

Analógica y digital



Humedad de tierra

Analógica y digital



Nivel de agua

Entrada analógica



DHT11

Temperatura y humedad

Entrada digital responde SÍ o NO Entrada analógica responde CUÁNTO (0 a 1023) Salida el Arduino la controla Digital especial envía datos

Hoy vamos a conocer cada uno: cómo funciona, cómo se conecta y cómo se programa.

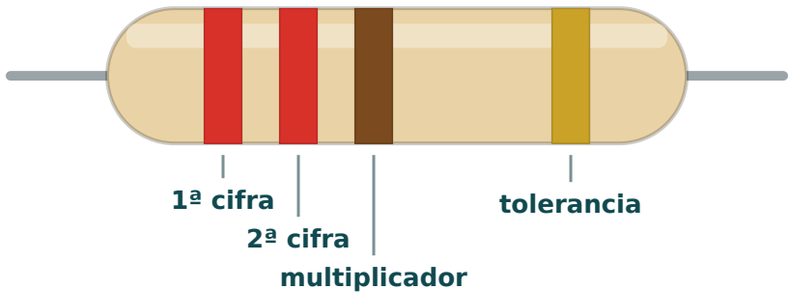
PLAN DE HOY

La ruta de la clase

1. **Herramientas**
   Código de colores de las resistencias y el Monitor Serie.
2. **Entradas básicas**
   Potenciómetro (a fondo) y pulsador.
3. **Salidas nuevas**
   Buzzer y display de 7 segmentos.
4. **Sensores**
   IR, sonido, humedad de tierra, nivel de agua y DHT11.
5. **Retos**
   Combina sensores y salidas para crear tus propios inventos.

El código de colores

Resistencia de 4 bandas



Las resistencias son tan pequeñas que no cabe el número impreso. Por eso su valor se escribe **con colores**.

Truco para recordar el orden: los colores del 2 al 7 siguen el arcoíris: **rojo, naranja, amarillo, verde, azul, violeta**. Antes van negro (0) y café (1); después gris (8) y blanco (9).

Color	Cifra	Multiplicador	Tolerancia
Negro	**0**	× 1	
Café (marrón)	**1**	× 10	± 1 %
Rojo	**2**	× 100	± 2 %
Naranja	**3**	× 1.000	
Amarillo	**4**	× 10.000	
Verde	**5**	× 100.000	
Azul	**6**	× 1.000.000	
Violeta	**7**		
Gris	**8**		
Blanco	**9**		
Dorado	—	× 0,1	± 5 %
Plateado	—	× 0,01	± 10 %

Cómo leer una resistencia de 4 bandas



rojo 2 · rojo 2 · café ×10

22 × 10 = 220 Ω → para LEDs



café 1 · negro 0 · rojo ×100

10 × 100 = 1.000 Ω = 1 kΩ



café 1 · negro 0 · naranja ×1.000

10 × 1.000 = 10.000 Ω = 10 kΩ

Paso a paso

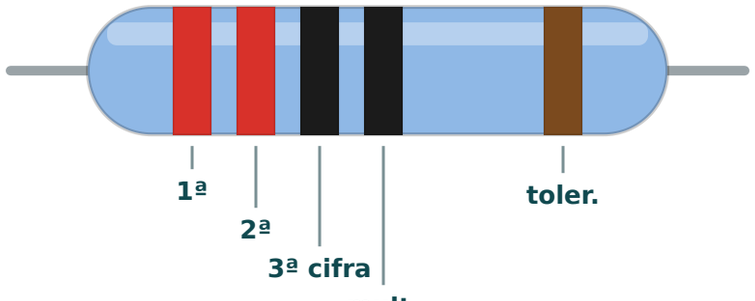
1. Pon la banda **dorada** a la derecha.
2. Las 2 primeras bandas son **cifras**: escríbelas juntas.
3. La 3ª banda dice **cuántos ceros** agregar (multiplicador).
4. La última es la **tolerancia**: cuánto se puede equivocar (±5 %).

k = kilo = 1.000. Por eso 10.000 Ω se escribe 10 kΩ y 4.700 Ω se escribe 4,7 kΩ.

Truco: la 3ª banda dice cuántos ceros poner. Café = 1 cero, rojo = 2 ceros, naranja = 3 ceros.

¿Y si tiene 5 bandas? ¿Por dónde empiezo?

Resistencia de 5 bandas (cuerpo azul)



2 · 2 · 0 × 1 = **220 Ω**

Con **5 bandas** hay **3 cifras** antes del multiplicador. Son más precisas (±1 %, banda café). Muchos kits traen de este tipo.

➡️ La dirección

La banda de tolerancia (dorada, plateada o café) está **un poco más separada** de las demás. Ponla a la derecha y lee de izquierda a derecha.

🚫 Nunca empieza así

Una resistencia **nunca empieza** por dorado o plateado. Si ves esos colores a la izquierda, voltéala.

📦 Las de tu kit

220 Ω: para LEDs y el display.
10 kΩ: para botones y sensores.
1 kΩ: usos varios.

🔎 ¿Dudas?

Si los colores se ven raros con la luz del salón, mídela con un **multímetro** en modo Ω. Es la forma segura.

¿Cuánto vale cada resistencia?

A



¿ _____ Ω ?

B



¿ _____ Ω ?

C



¿ _____ Ω ?

D



¿ _____ Ω ?

En parejas

Usen la tabla de colores y escriban el valor de cada una en su cuaderno.

⏱ Tienen **3 minutos**.

⭐ **Bonus:** busquen en su kit una resistencia de 220 Ω y otra de 10 kΩ.

Respuestas del reto

A



4 · 7 · ×100

4.700 Ω = 4,7 kΩ

B



3 · 3 · ×10

330 Ω

C



5 · 6 · ×1

56 Ω

D



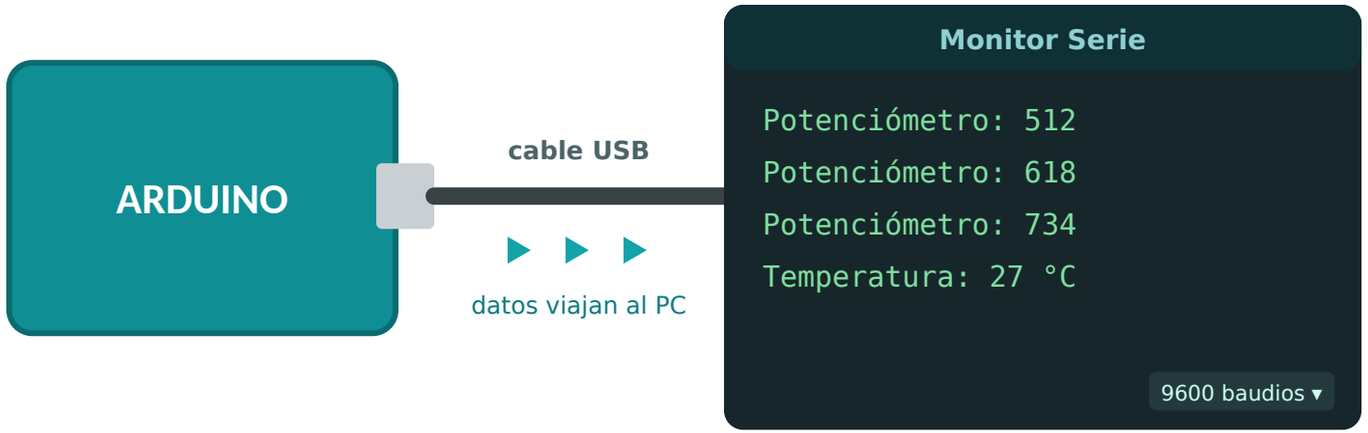
1 · 0 · 0 · ×1.000 (5 bandas)

100.000 Ω = 100 kΩ

El error más común es **olvidar el multiplicador**. Las dos primeras bandas son cifras; la tercera dice **cuántos ceros** agregar.

En la **D** hay 5 bandas: tres cifras (1, 0, 0) y luego el multiplicador naranja (× 1.000).

El Monitor Serie: los ojos dentro del Arduino



Un sensor da números, pero el Arduino no tiene pantalla. El **Monitor Serie** muestra en el computador lo que el Arduino “piensa” y “siente”.

🔍 ¿Cómo lo abro?

En el Arduino IDE, el ícono de la **lupa** arriba a la derecha, o **Ctrl + Shift + M**.

⚡ Regla de oro

La velocidad elegida en el monitor (abajo a la derecha) debe ser **igual** a la del código: `Serial.begin(9600)`. Si no, verás símbolos raros.

🔌 Usa los pines 0 y 1

La comunicación serie viaja por los pines 0 (RX) y 1 (TX). Por eso no conectamos nada ahí.

Tu primer mensaje

HolaSerial.ino

```ino
int contador = 0;            // variable que vamos a mostrar

void setup() {
  Serial.begin(9600);        // abre la comunicación
  Serial.println("¡Hola! Soy tu Arduino");
}

void loop() {
  Serial.print("Contador: ");  // escribe SIN bajar de línea
  Serial.println(contador);    // escribe el número y SÍ baja
  contador = contador + 1;     // suma 1
  delay(1000);                 // espera 1 segundo
}
```

Monitor Serie

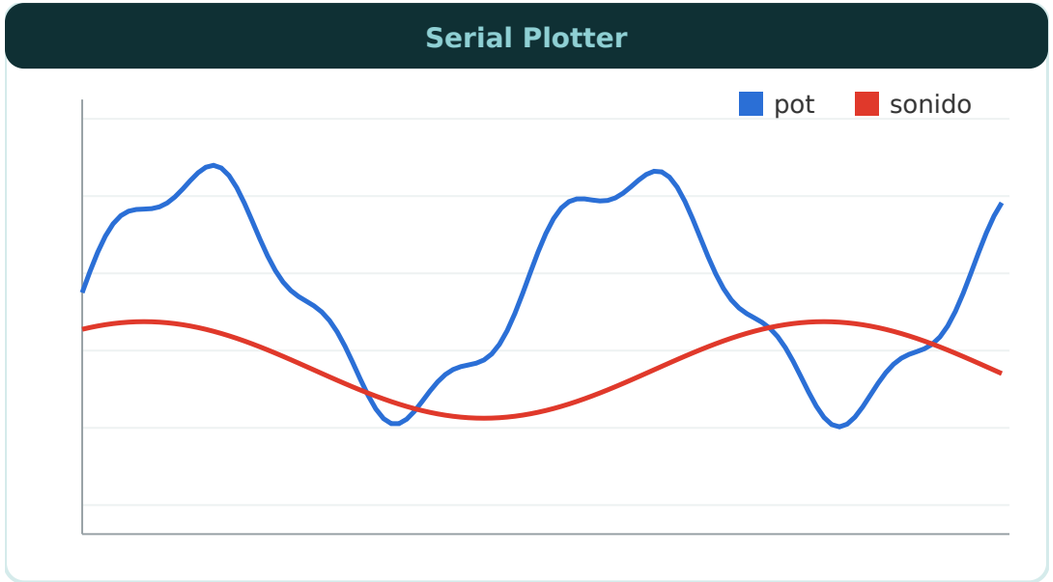
```
¡Hola! Soy tu Arduino
Contador: 0
Contador: 1
Contador: 2
Contador: 3
Contador: 4
_
```

`Serial.print()` escribe y se queda en la **misma línea**.

`Serial.println()` escribe y **baja** a la siguiente línea (ln = line).

Texto entre **comillas** se muestra tal cual. Una variable **sin comillas** muestra su valor.

El Serial Plotter: ver los datos como una gráfica



Ábrelo en **Herramientas ▸ Serial Plotter** o con **Ctrl + Shift + L**. Usa el mismo código que el Monitor Serie, pero dibuja los números como una línea que se mueve.

Plotter.ino

```ino
void setup() {
  Serial.begin(9600);
}

void loop() {
  int pot = analogRead(A0);
  int sonido = analogRead(A1);

  // formato "nombre:valor", separados por coma
  Serial.print("pot:");
  Serial.print(pot);
  Serial.print(",");
  Serial.print("sonido:");
  Serial.println(sonido);  // println al final
  delay(50);
}
```

Ideal para sensores que cambian rápido, como el de sonido o el potenciómetro.

¿Qué es un potenciómetro?



Es una **resistencia que puedes cambiar girando una perilla**. Para el Arduino es la forma más fácil de leer un valor analógico: girar = cambiar un número entre 0 y 1023.

🔊 **Volumen** de un parlante o radio

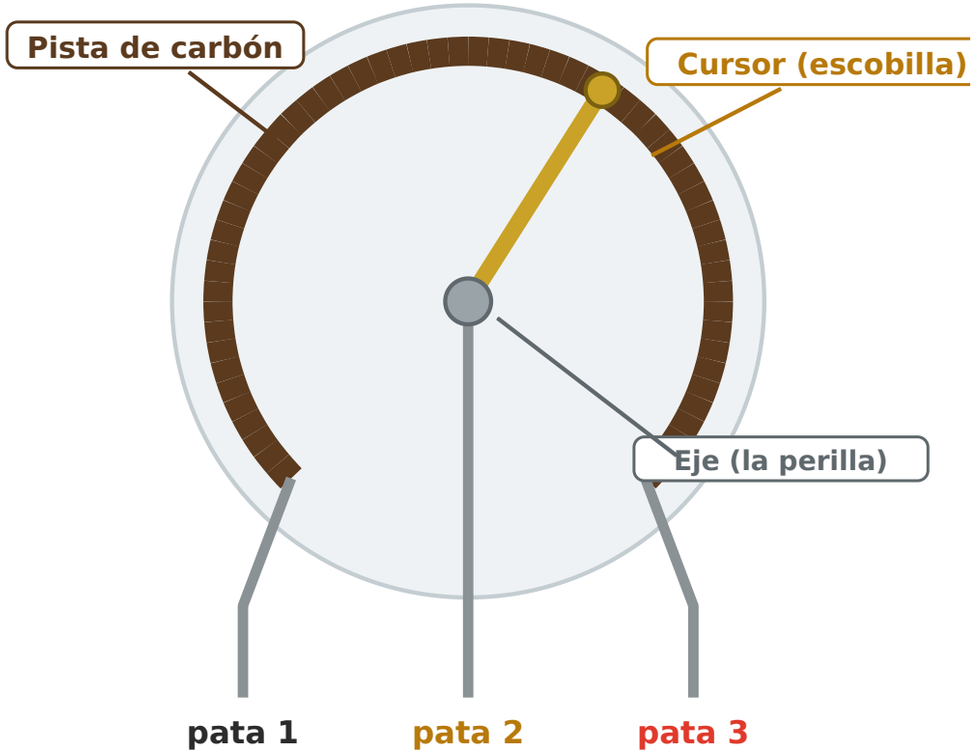
🌀 **Velocidad** de un ventilador

🎮 **Joystick** de un control (¡son 2 potenciómetros!)

¿Qué dice “B10K”? **10K** = 10.000 Ω entre las dos patas de los extremos. **B** = lineal: si giras la mitad, cambia la mitad.

Tiene 3 patas, y eso es lo que más confunde. En las siguientes diapositivas vemos por qué.

Por dentro: una pista y un cursor que se desliza



1 · La pista

Es una **tira de carbón** en forma de herradura. Toda la pista mide 10 kΩ. Sus dos puntas están soldadas a las **patas 1 y 3**.

2 · El cursor

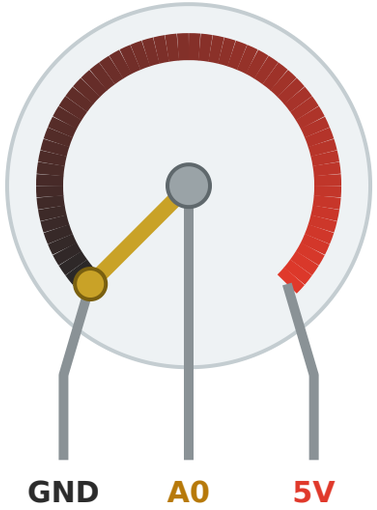
Es una lámina de metal que **toca la pista** y se mueve cuando giras la perilla. Está conectado a la **pata 2** (la del medio).

3 · La idea clave

Entre las patas 1 y 3 siempre hay 10 kΩ. Lo que cambia es **dónde toca el cursor**, y eso es lo que lee el Arduino por la pata del medio.

El divisor de voltaje: ¿qué voltaje toca el cursor?

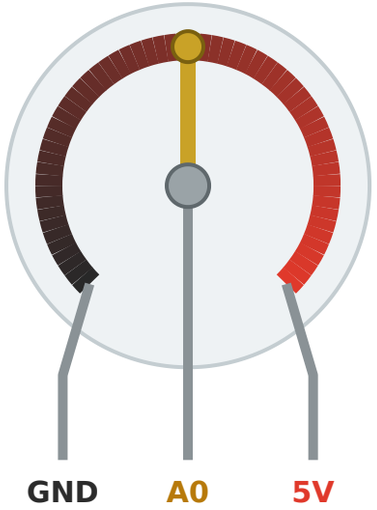
cursor junto a GND



0 V

```
analogRead = 0
```

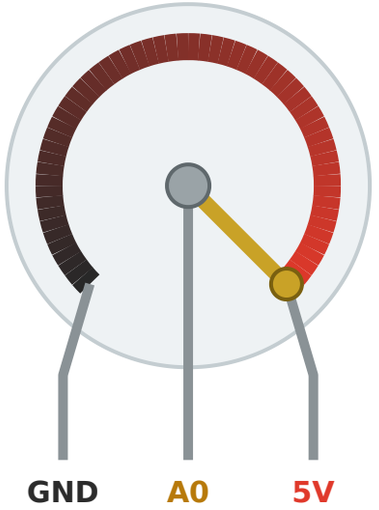
cursor en la mitad



2,5 V

```
analogRead = 512
```

cursor junto a 5V



5 V

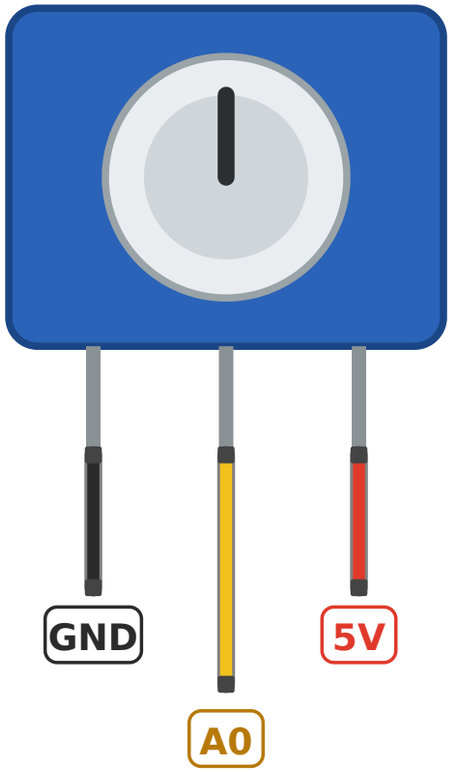
```
analogRead = 1023
```

Conectamos un extremo a **5V** y el otro a **GND**. Así la pista queda como una **rampa de voltaje**: 0 V en una punta y 5 V en la otra (fíjate en el color).

El cursor “recoge” el voltaje del punto donde está tocando. Ese voltaje sale por la pata del medio hacia **A0**.

Es como un **tobogán**: arriba hay 5 V, abajo 0 V. El cursor es un niño que se detiene en cualquier punto del tobogán.

Cómo conectarlo: cada pata tiene su trabajo



✅ La regla

Los **extremos** van a **5V** y a **GND**.
La pata del **medio** va al pin analógico (**A0**).

🔄 ¿Extremos al revés?

No pasa nada malo: solo **gira al revés** (a la derecha baja en vez de subir). Si quieres, intercambia los cables de los extremos.

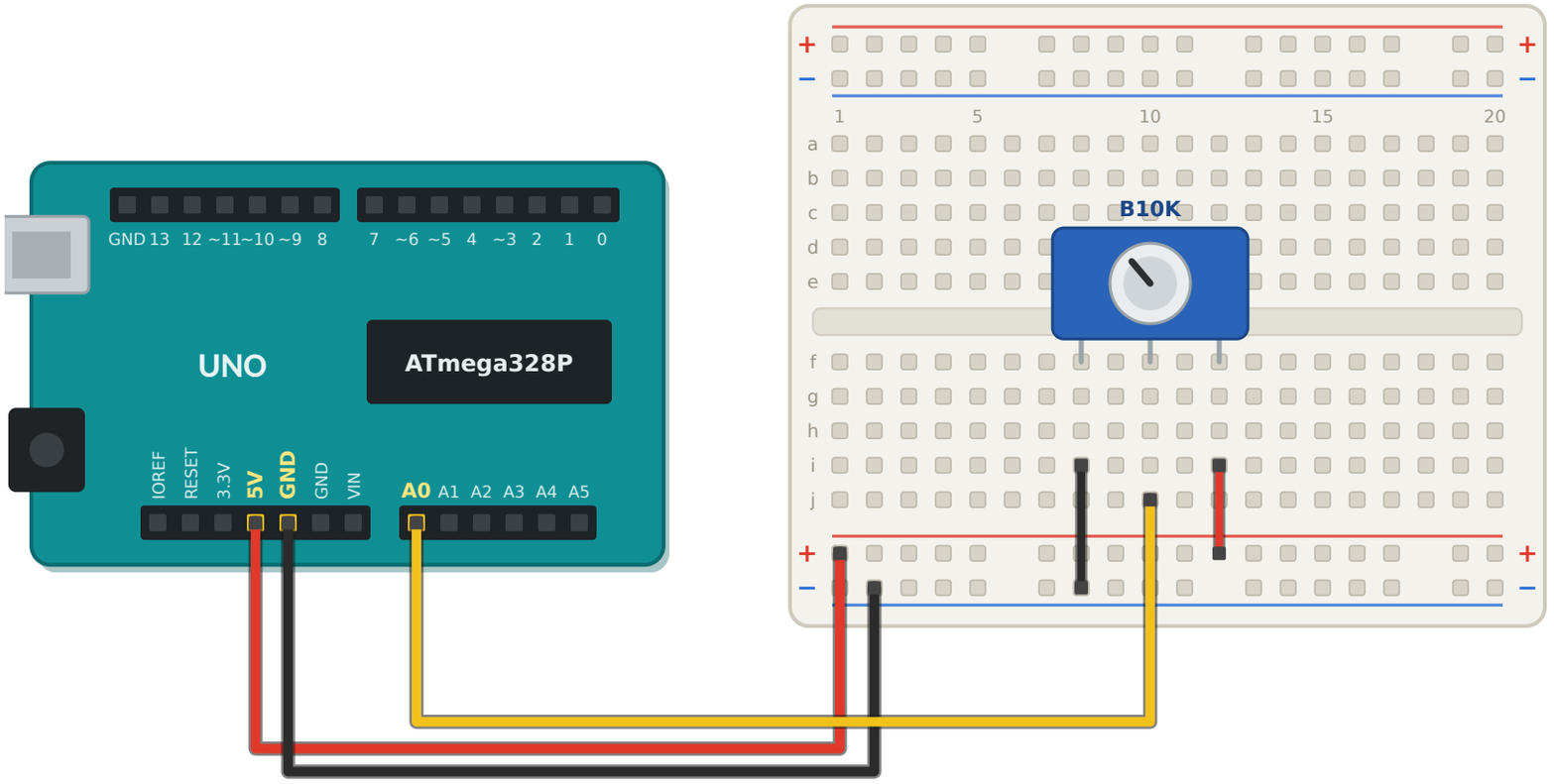
⛔ Nunca el medio a 5V o GND

Si conectas la pata del medio a 5V o a GND y giras hasta el tope, haces un **cortocircuito**: el potenciómetro se calienta y el Arduino se puede reiniciar.

🔎 ¿Cuál es la del medio?

Siempre la del **centro** de las tres, la que está sola entre las otras dos. Es la del cursor.

Montaje: el potenciómetro en A0



Conexiones

5V	→	riel + (rojo) de la protoboard
GND	→	riel − (azul) de la protoboard
Riel −	→	pata izquierda (columna 8)
A0	→	pata del MEDIO (columna 10)
Riel +	→	pata derecha (columna 12)

Recuerda: los 5 huecos de una columna están unidos. Cada pata del potenciómetro queda en su propia columna, y ahí conectamos su cable.

Código: leer el potenciómetro

LeerPotenciometro.ino

```ino
const int PIN_POT = A0;     // la pata del medio va a A0

void setup() {
  Serial.begin(9600);       // para ver los números
}

void loop() {
  int valor = analogRead(PIN_POT);        // de 0 a 1023
  float voltaje = valor * 5.0 / 1023.0;   // pasar a voltios

  Serial.print("Valor: ");
  Serial.print(valor);
  Serial.print("   Voltaje: ");
  Serial.print(voltaje);
  Serial.println(" V");
  delay(200);
}
```

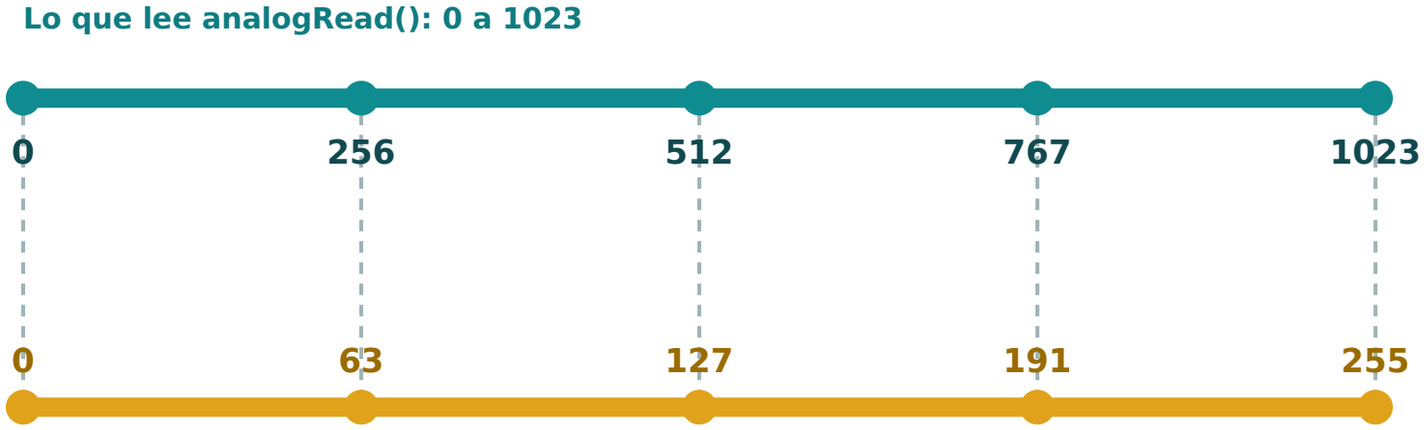
Monitor Serie

```
Valor: 0     Voltaje: 0.00 V
Valor: 204   Voltaje: 1.00 V
Valor: 512   Voltaje: 2.50 V
Valor: 860   Voltaje: 4.20 V
Valor: 1023  Voltaje: 5.00 V
```

Prueba: gira la perilla despacio y mira cómo cambian los números. Luego ábrelo en el **Serial Plotter**.

¿Por qué 5.0 y no 5? Con el punto decimal, el Arduino calcula con decimales (`float`). Sin él, perderías los decimales.

map(): traducir de una escala a otra



Es como pasar de **centímetros a pulgadas**: el mismo punto, contado en otra regla. Si el potenciómetro está a la mitad (512), el LED queda a la mitad (127).

La receta

```
map(valor,
    // de esta escala…
    0, 1023,
    // …a esta otra
    0, 255);
```

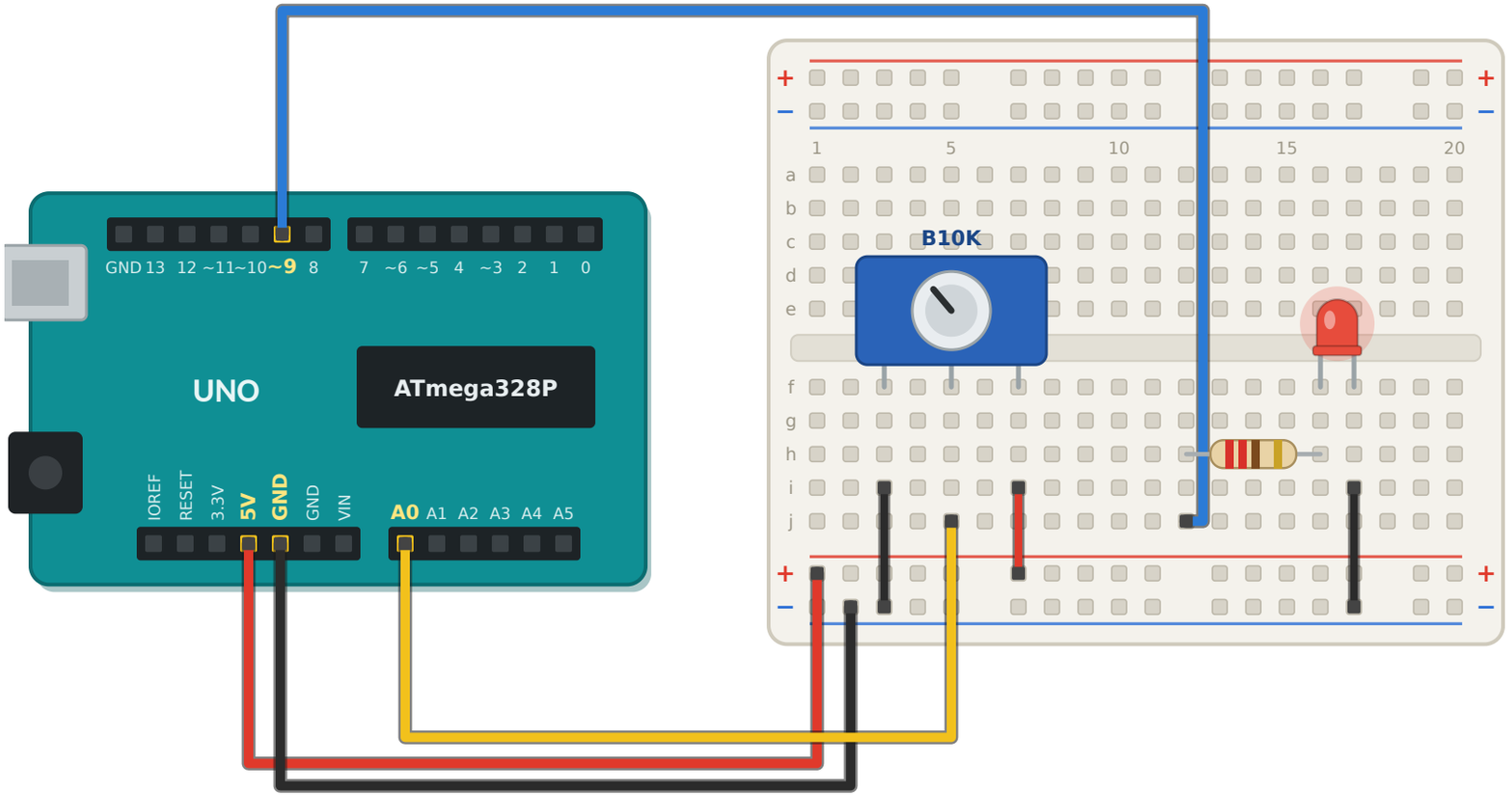
Más ejemplos:

`map(valor, 0, 1023, 0, 100)` → porcentaje

`map(valor, 0, 1023, 0, 9)` → un número para el display

Ojo: map() trabaja con números enteros: corta los decimales.

Montaje: el potenciómetro controla un LED



Conexiones

Riel −	→	pata izquierda del pot (col. 3)
A0	→	pata del medio del pot (col. 5)
Riel +	→	pata derecha del pot (col. 7)
Pin ~9	→	resistencia de 220 Ω (col. 12)
220 Ω	→	de la col. 12 a la pata larga del LED
Riel −	→	pata corta del LED (col. 17)

El LED va en el pin **~9** porque necesitamos **PWM** para los brillos intermedios (clase 2).

Código: la perilla controla el brillo

PotLED.ino

```ino
const int PIN_POT = A0;
const int LED = 9;            // pin con ~ (PWM)

void setup() {
  pinMode(LED, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int valor = analogRead(PIN_POT);           // 0 a 1023
  int brillo = map(valor, 0, 1023, 0, 255);  // 0 a 255
  analogWrite(LED, brillo);

  Serial.print("Pot: ");
  Serial.print(valor);
  Serial.print("  ->  Brillo: ");
  Serial.println(brillo);
  delay(50);
}
```

1 · Leer

`analogRead()` nos da la posición de la perilla: 0 a 1023.

2 · Traducir

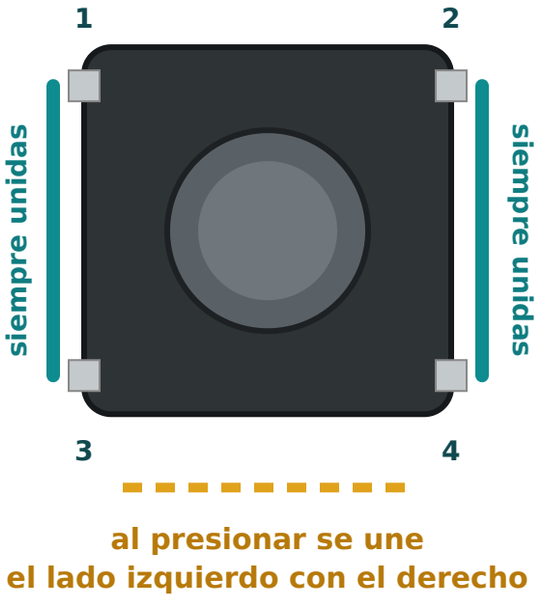
`map()` convierte ese número a la escala del LED: 0 a 255.

3 · Actuar

`analogWrite()` enciende el LED con ese brillo.

Leer → traducir → actuar. Casi todos los proyectos con sensores siguen esta receta.

El pulsador por dentro: 4 patas, 2 pares



🔗 Las patas van en pares

Las patas 1 y 3 están **siempre unidas** por dentro. Las patas 2 y 4 también. Por eso, aunque tenga 4 patas, el botón funciona como un interruptor de 2.

👇 Al presionar

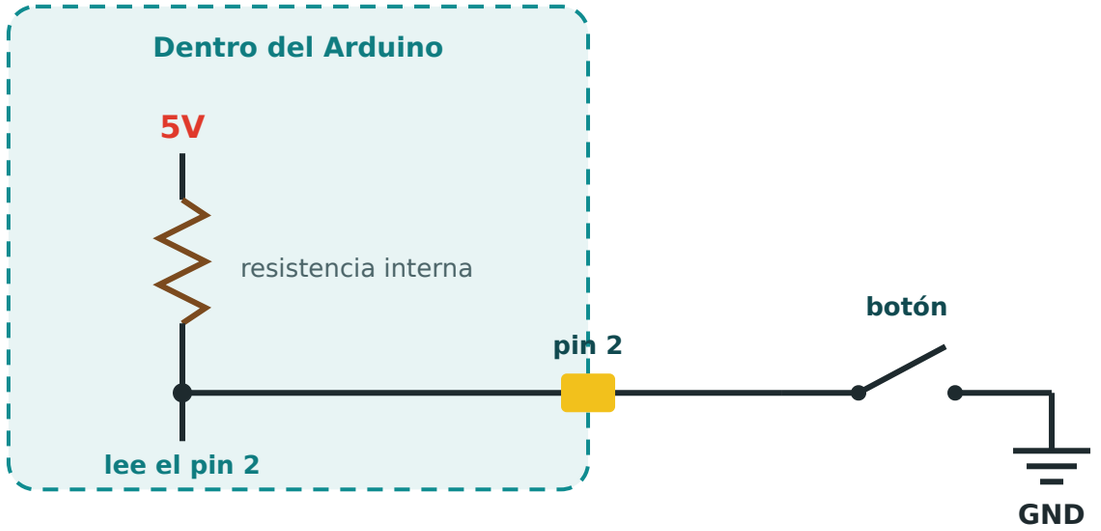
Se une el par de la izquierda con el de la derecha y la corriente puede pasar.

🧩 En la protoboard

Ponlo **“a caballo” sobre el canal del centro**. Así cada par queda en su propia columna y no hay cortos.

Si el botón “siempre está presionado”, seguramente lo pusiste girado 90°. Gíralo y prueba de nuevo.

INPUT_PULLUP: el botón sin resistencia extra



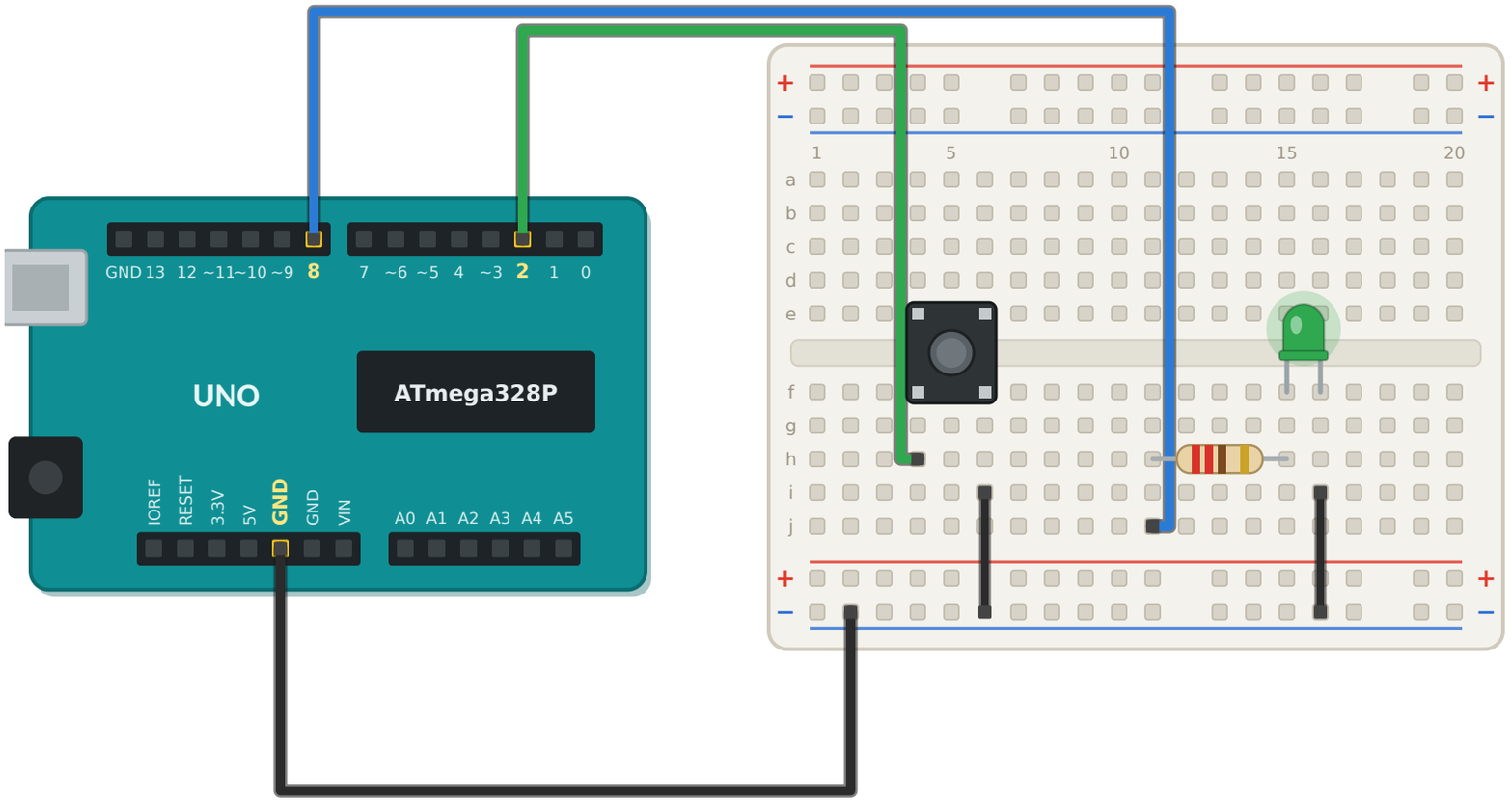
Botón	¿Qué pasa?	digitalRead()
Suelto	La resistencia interna lleva el pin a 5V	**HIGH (1)**
Presionado	El botón une el pin con GND	**LOW (0)**

¡La lógica queda al revés!
Presionado = **LOW**. Por eso en el código preguntamos `if (estado == LOW)`.

Conexión: una pata del botón al **pin 2** y la otra a **GND**. Nada más.

En la clase 1 usamos una resistencia de 10 kΩ (pull-down). El Arduino trae una **resistencia interna** que podemos activar con `pinMode(2, INPUT_PULLUP)`. ¡Un componente menos!

Montaje: botón en el pin 2 y LED en el pin 8



Conexiones

GND	→	riel − de la protoboard
Pin 2	→	pata izquierda del botón (col. 4)
Riel −	→	pata derecha del botón (col. 6)
Pin 8	→	resistencia de 220 Ω (col. 11)
220 Ω	→	de la col. 11 a la pata larga del LED
Riel −	→	pata corta del LED (col. 16)

El botón va **sobre el canal**. Gracias a `INPUT_PULLUP` no necesita resistencia ni cable a 5V.

Código: el botón enciende el LED

BotonLED.ino

```ino
const int BOTON = 2;
const int LED = 8;

void setup() {
  pinMode(BOTON, INPUT_PULLUP);   // resistencia interna
  pinMode(LED, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int estado = digitalRead(BOTON);
  Serial.println(estado);          // 1 = suelto, 0 = presionado

  if (estado == LOW) {             // ¿está presionado?
    digitalWrite(LED, HIGH);
  } else {
    digitalWrite(LED, LOW);
  }
  delay(50);
}
```

Abre el Monitor Serie: verás una lluvia de **1**. Presiona el botón y cambian a **0**.

💡 Reto mini

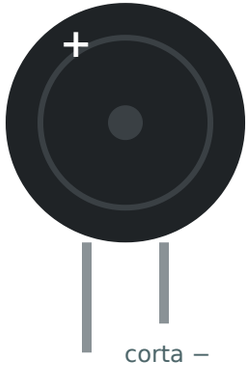
Cambia el código para que el LED quede **encendido cuando NO presionas** y se apague al presionar.

⭐ Reto plus: interruptor

Que cada vez que presiones, el LED **cambie de estado** (si estaba apagado se enciende, y al revés). Pista: guarda el estado en una variable `bool`.

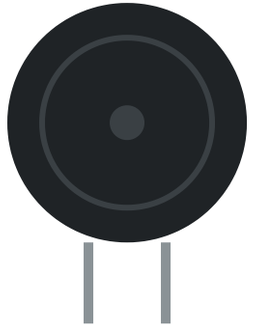
El buzzer: activo o pasivo

ACTIVO (el tuyo)



abajo: sellado (negro)

PASIVO



abajo: se ve la placa verde

Tu kit trae el TMB12A05: es un buzzer **activo** de 5 V. Tiene su propio “oscilador” por dentro: solo hay que darle energía y suena.

	Buzzer ACTIVO	Buzzer PASIVO
¿Cómo suena?	Siempre el mismo tono (un “biiip”)	Puede hacer notas y melodías
¿Cómo se controla?	`digitalWrite(pin, HIGH)`	`tone(pin, frecuencia)`
Por debajo	Sellado con resina negra	Se ve la placa verde
Prueba rápida	Lo conectas a 5V y GND y **suena solo**	Con 5V fijo solo hace “clic”

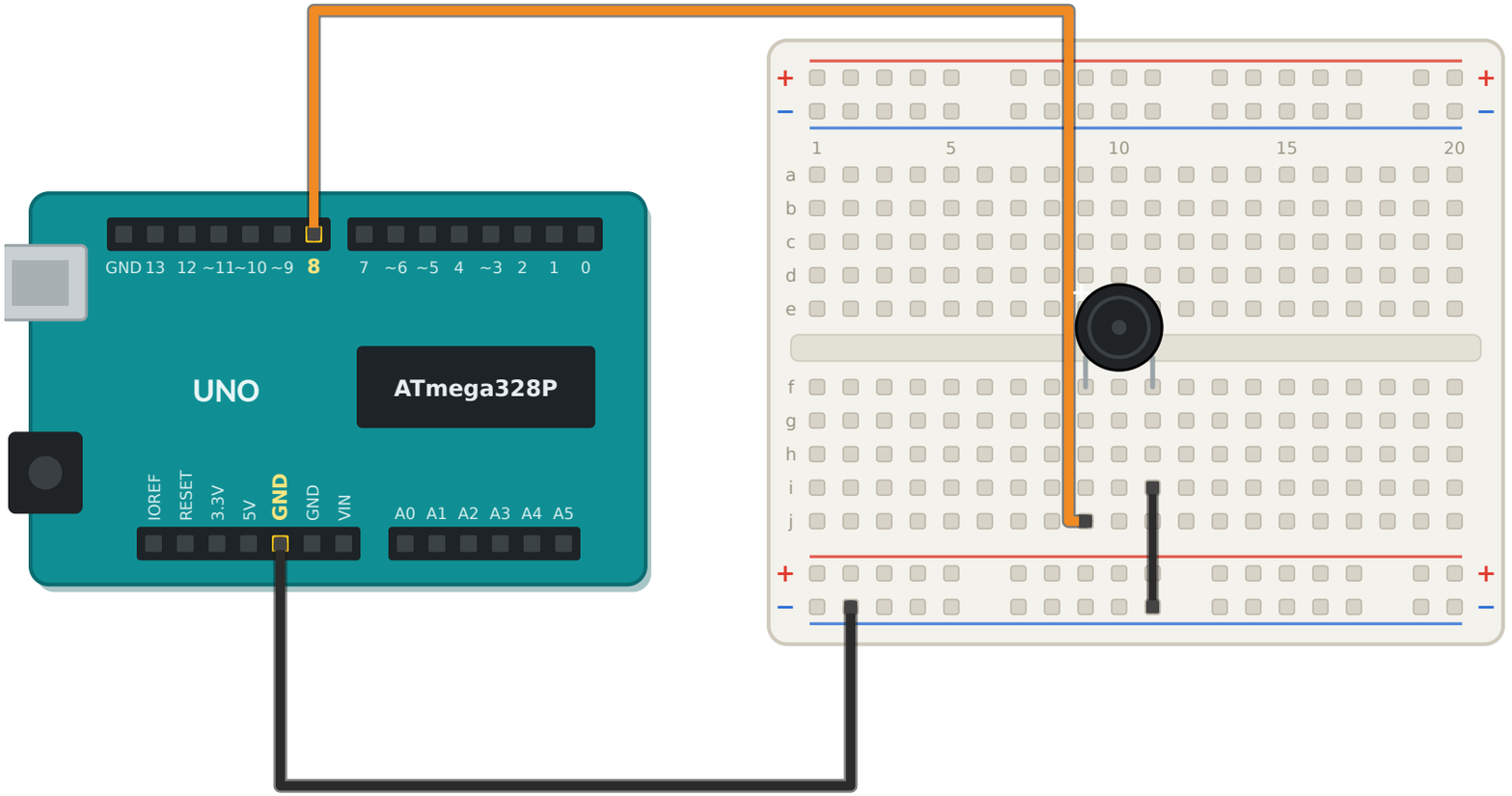
➕ Tiene polaridad

La pata **larga** (+) va al pin del Arduino y la corta (−) a GND. Busca el “+” impreso arriba.

🔇 La etiqueta

Si trae un sticker encima, quítalo: está ahí para la fábrica y hace que suene más bajito.

Montaje: buzzer activo en el pin 8



Conexiones

Pin 8	→	pata larga (+) del buzzer (col. 9)
GND	→	riel − de la protoboard
Riel −	→	pata corta (−) del buzzer (col. 11)

Este buzzer consume poca corriente (unos 30 mA), así que lo conectamos **directo al pin**, sin resistencia.

Si quedó al revés, **no suena**, pero tampoco se daña. Gíralo y prueba otra vez.

Código: pitidos con el buzzer activo

Buzzer.ino

```ino
const int BUZZER = 8;

void setup() {
  pinMode(BUZZER, OUTPUT);
}

void loop() {
  // tres pitidos cortos
  for (int i = 0; i < 3; i++) {
    digitalWrite(BUZZER, HIGH);   // suena
    delay(100);
    digitalWrite(BUZZER, LOW);    // silencio
    delay(100);
  }
  delay(1500);                    // pausa larga
}
```

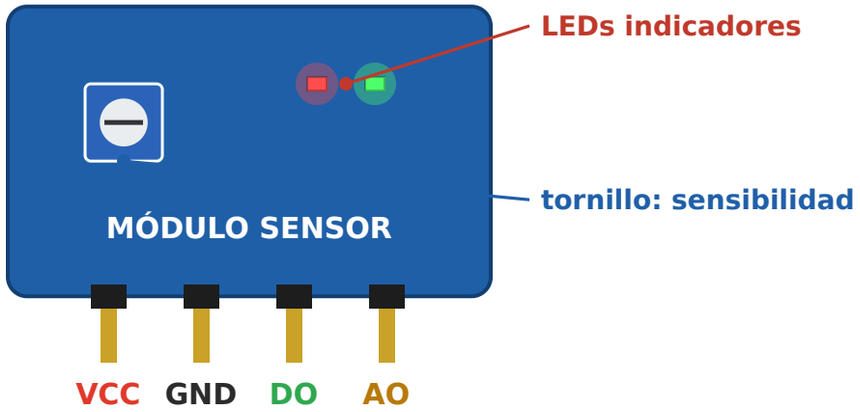
SiFueraPasivo.ino

```ino
// Solo si tu buzzer fuera PASIVO:
tone(8, 440);    // nota LA (440 Hz)
delay(500);
noTone(8);       // apagar el sonido
delay(500);
```

Con el activo, el ritmo lo hacemos con delay(): cuánto suena y cuánto calla. Así se pueden hacer alarmas, avisos y hasta código Morse.

Ojo con los oídos del salón: prueben con delays cortos y no lo dejen sonando todo el tiempo.

Antes de empezar: cómo leer los pines de un módulo



Si dice…	Significa	Va a…
VCC · + · 5V	Alimentación	5V (riel +)
GND · G · −	Tierra	GND (riel −)
DO · OUT · S	Salida **digital**: SÍ o NO	Un pin digital (2 a 13)
AO · A0 · S	Salida **analógica**: CUÁNTO	Un pin analógico (A0 a A5)

El orden de los pines cambia según el fabricante.
Antes de conectar, lee siempre las letras impresas en tu módulo.

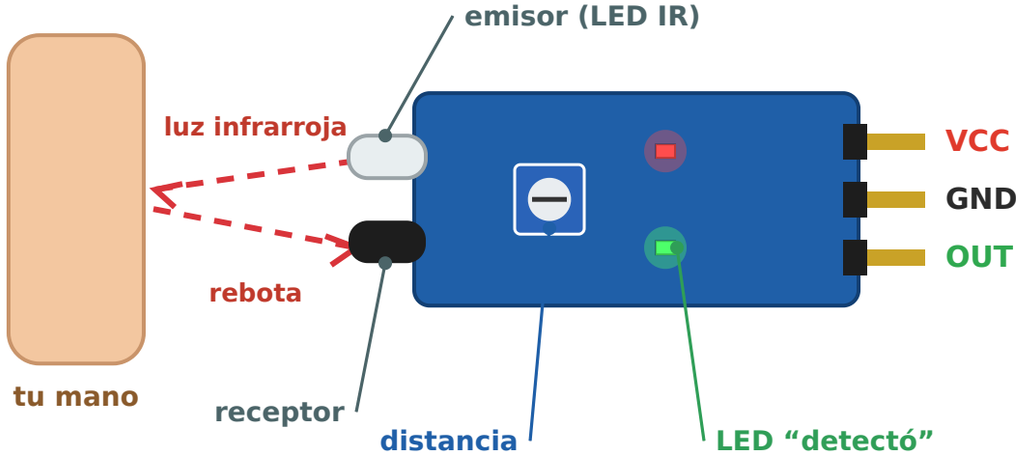
⚠️ VCC y GND al revés

Es el error que más daña sensores. Revisa dos veces antes de conectar el USB.

🔧 El tornillo azul

Ajusta **cuándo** se activa la salida digital (DO). No cambia la analógica (AO).

Sensor infrarrojo de obstáculos



📱 **Truco:** la luz infrarroja es invisible para nosotros, pero muchas cámaras de celular sí la ven. Apunta la cámara al LED transparente y verás un brillo morado.

¿Cómo funciona?

El LED transparente lanza luz infrarroja. Si hay un objeto cerca, la luz **rebota** y llega al receptor negro. Entonces el módulo avisa por el pin **OUT**.

¡La salida es al revés!

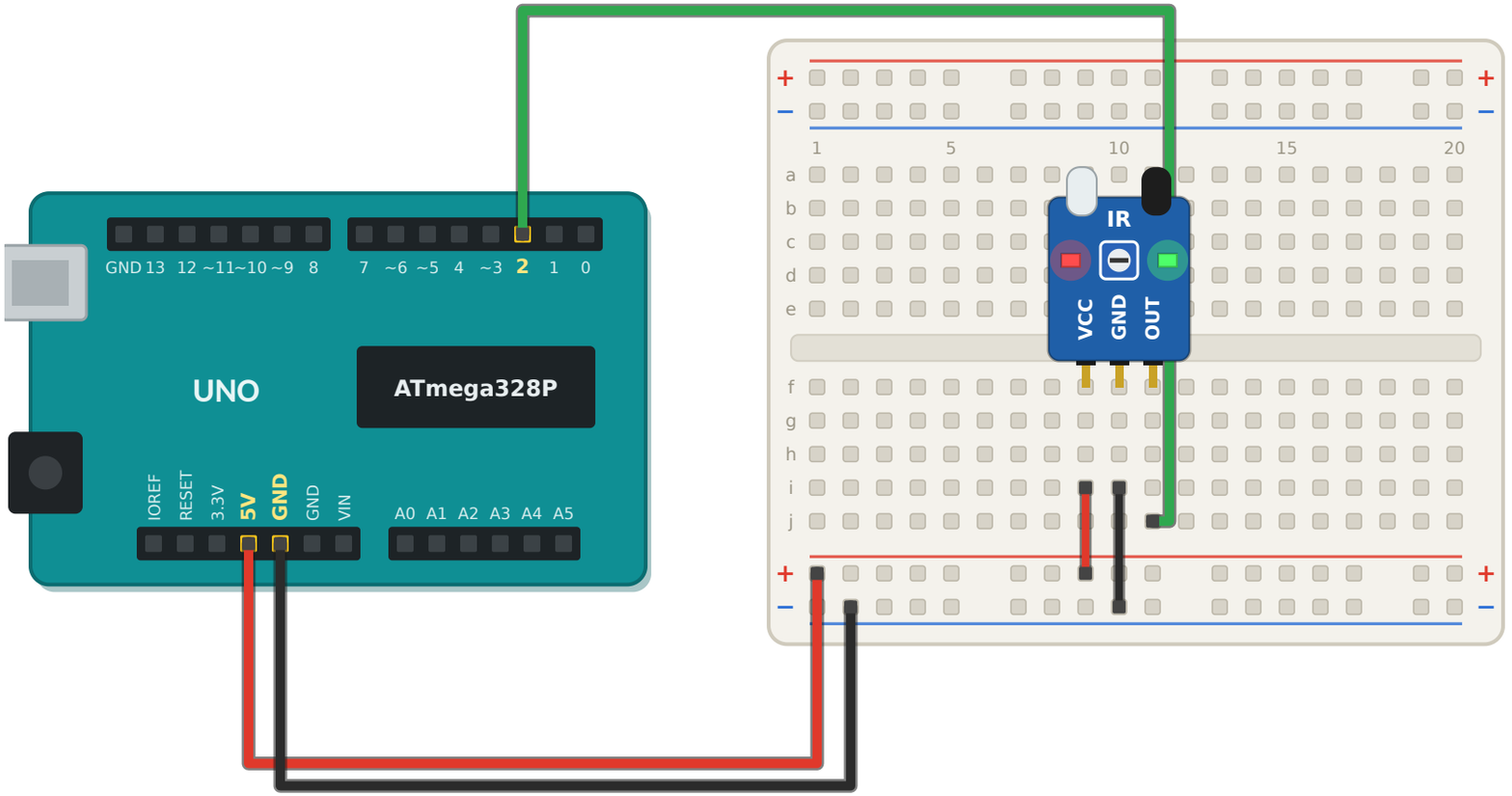
Hay obstáculo → OUT = **LOW** (0).
No hay nada → OUT = **HIGH** (1).

Alcance y límites

De 2 a 30 cm, según el tornillo azul. Le cuesta ver objetos **negros** (absorben la luz) y se confunde con el **sol directo**.

Se usa en robots que esquivan paredes, dispensadores automáticos de jabón y contadores de objetos.

Montaje: sensor IR en el pin 2



Conexiones

5V	→	riel + de la protoboard
GND	→	riel − de la protoboard
Riel +	→	VCC del sensor (col. 9)
Riel −	→	GND del sensor (col. 10)
Pin 2	→	OUT del sensor (col. 11)

Como aviso usamos el **LED “L”** que trae la placa, conectado internamente al pin 13. ¡No hay que cablear nada más!

Código: detectar un obstáculo

SensorIR.ino

```ino
const int PIN_IR = 2;
const int LED = 13;          // LED "L" de la placa

void setup() {
  pinMode(PIN_IR, INPUT);
  pinMode(LED, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int estado = digitalRead(PIN_IR);

  if (estado == LOW) {       // LOW = hay algo cerca
    digitalWrite(LED, HIGH);
    Serial.println("¡Obstáculo detectado!");
  } else {
    digitalWrite(LED, LOW);
    Serial.println("Libre");
  }
  delay(200);
}
```

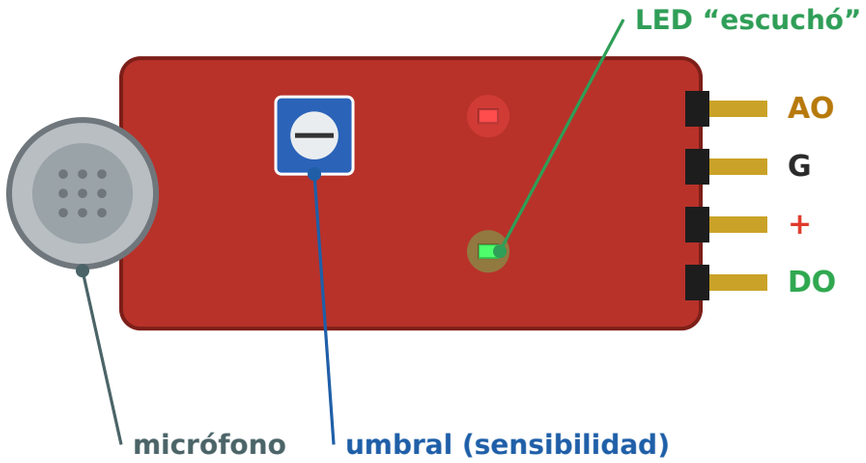
Para calibrar: pon tu mano a la distancia que quieres detectar y gira el tornillo azul hasta que el LED verde del módulo apenas se encienda.

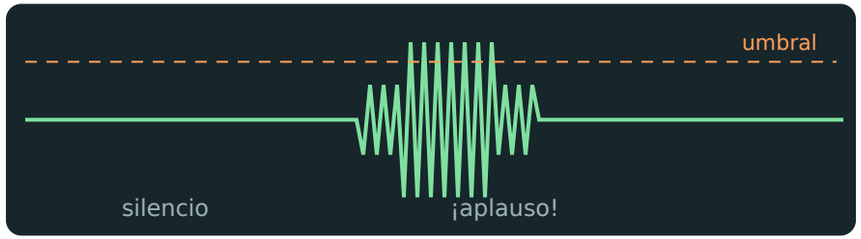
🧪 Experimenta

Prueba con una hoja blanca y con una negra. ¿A qué distancia detecta cada una? ¿Por qué crees que pasa eso?

Si el LED queda **siempre encendido**, el sensor está demasiado sensible: gira el tornillo hacia el otro lado.

Sensor de sonido: un micrófono que mide el ruido





¿Qué mide?

Qué tan **fuerte** es el sonido. No entiende palabras ni notas: solo “silencio” o “¡ruido!”.

AO: salida analógica

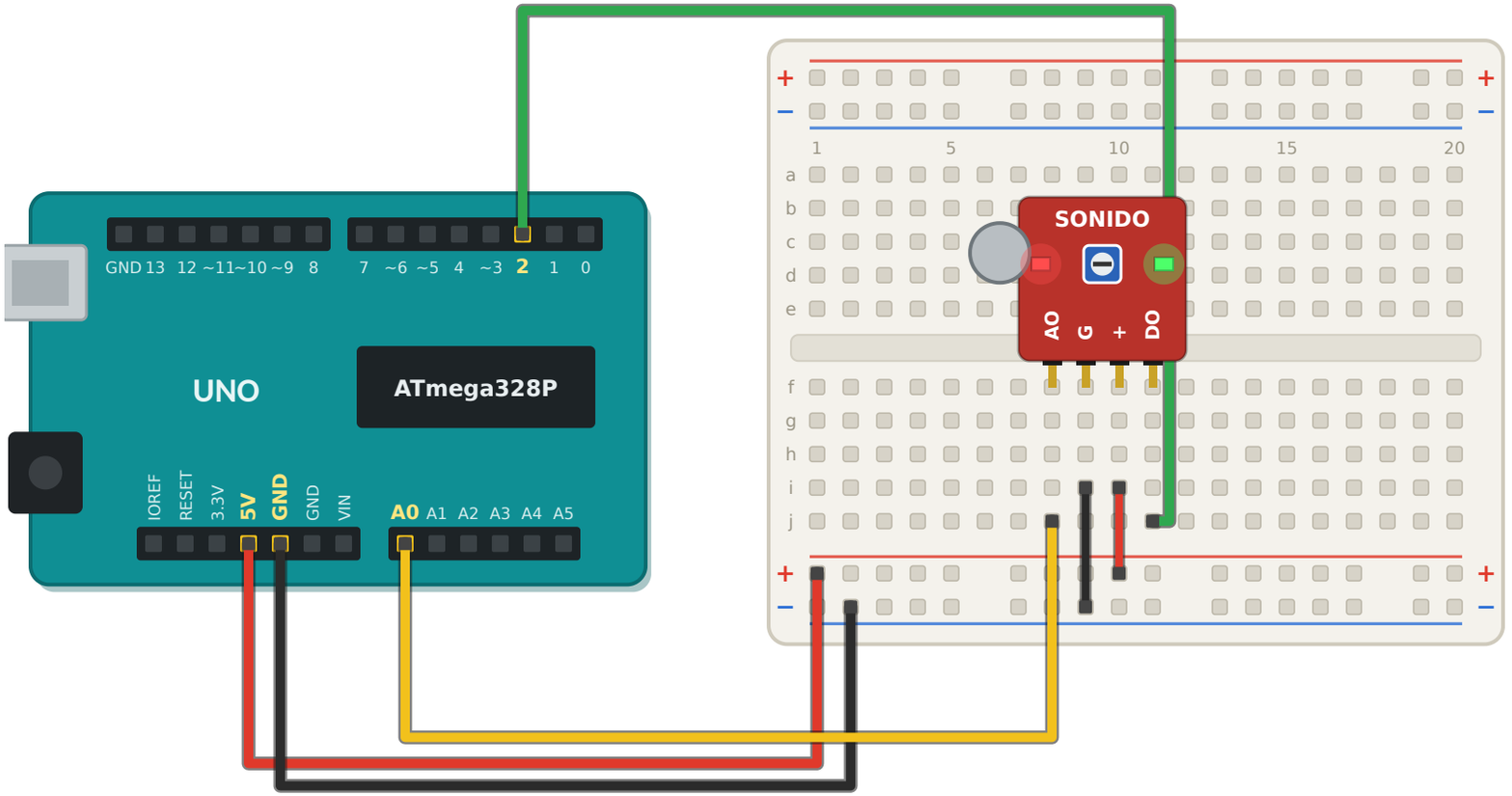
Da un número con **analogRead()**. En silencio se queda casi quieto; con un ruido fuerte **salta** hacia arriba y hacia abajo (mira la gráfica).

DO: salida digital

Se pone en **HIGH** cuando el ruido pasa el **umbral** que eliges con el tornillo azul. Ideal para detectar aplausos.

Calibra en silencio: gira el tornillo hasta que el LED “escuchó” se apague justo. Luego aplaude y debería parpadear.

Montaje: sensor de sonido en A0 y en el pin 2



Conexiones

A0	→	AO del sensor (col. 8)
Riel −	→	G del sensor (col. 9)
Riel +	→	+ del sensor (col. 10)
Pin 2	→	DO del sensor (col. 11)
5V / GND	→	a los rieles + y −

Usamos **las dos salidas**: AO para ver la onda en el Serial Plotter y DO para detectar el aplauso.

Código: el Arduino escucha un aplauso

SensorSonido.ino

```ino
const int PIN_AO = A0;      // salida analógica
const int PIN_DO = 2;       // salida digital
const int LED = 13;         // LED "L" de la placa

void setup() {
  pinMode(PIN_DO, INPUT);
  pinMode(LED, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int nivel = analogRead(PIN_AO);
  Serial.println(nivel);     // míralo en el Serial Plotter

  if (digitalRead(PIN_DO) == HIGH) {   // ¿pasó el umbral?
    digitalWrite(LED, HIGH);
    delay(1000);                       // encendido 1 segundo
    digitalWrite(LED, LOW);
  }
}
```

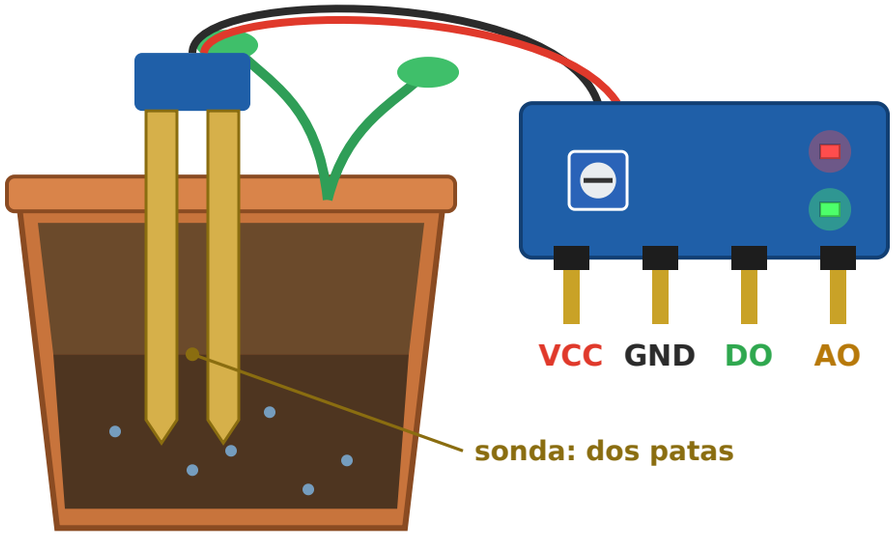
Abre el **Serial Plotter** (Ctrl + Shift + L), habla cerca del micrófono y luego aplaude. ¿Ves la diferencia?

¿El LED se queda prendido siempre? Hay dos opciones: el umbral está muy bajo (gira el tornillo) o tu módulo funciona al revés (cambia `HIGH` por `LOW` en el if).

🧪 Experimenta

¿Cuál es el número más alto que logra tu pareja con un aplauso? ¿Y con un silbido?

Sensor de humedad de tierra: ¿mi planta tiene sed?



¿Cómo funciona?

El agua **conduce la electricidad**. Las dos patas se clavan en la tierra: si está húmeda, la corriente pasa fácil entre ellas; si está seca, le cuesta.

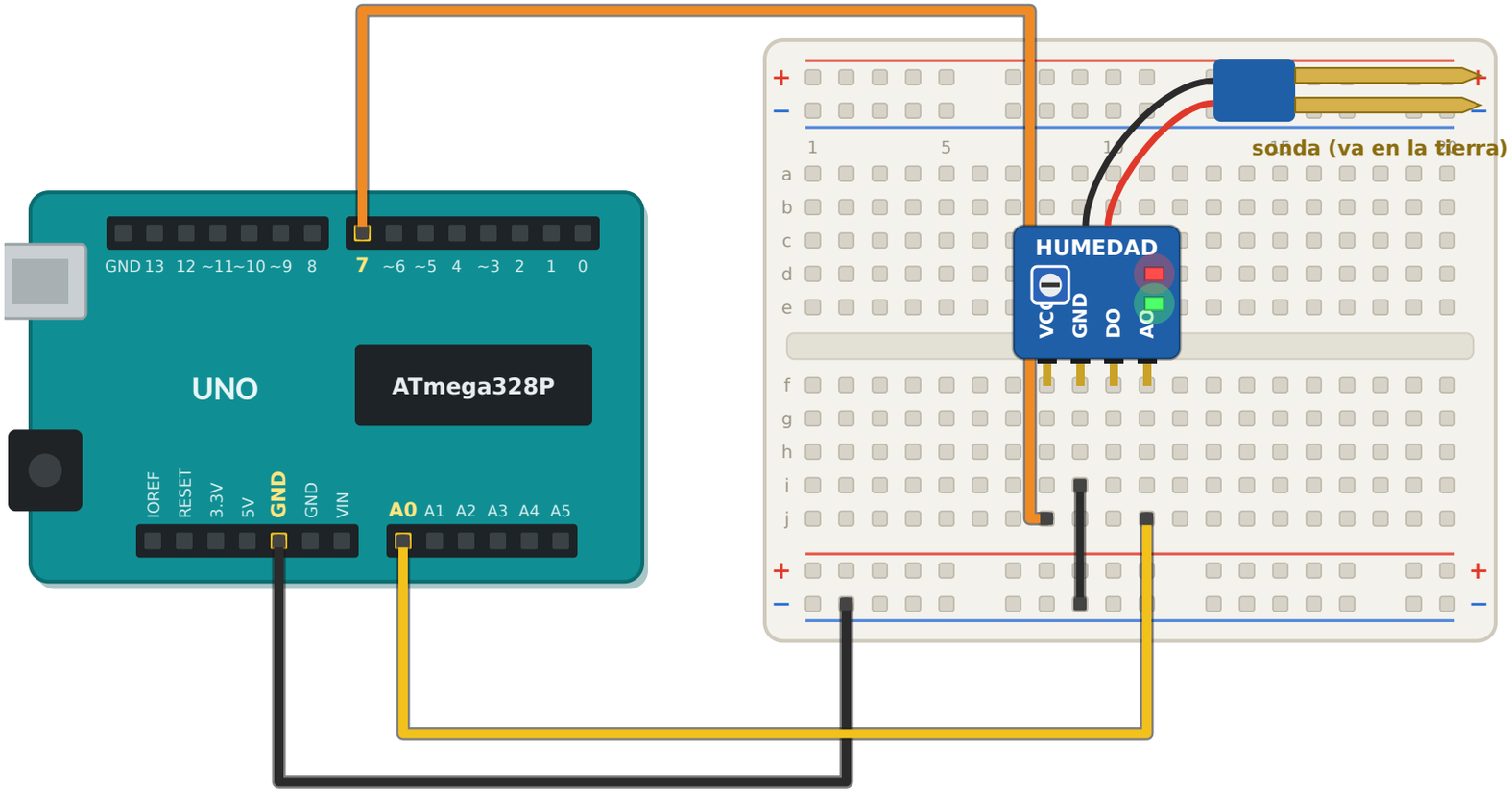
Los números van al revés

Seca → número **alto** (cerca de 1000).
Mojada → número **bajo** (300 a 500).
Cada sensor es distinto: hay que **calibrarlo**.

⚠️ Se oxida

Si le damos energía todo el tiempo, las patas se corroen en pocos días. Truco: lo alimentamos desde un **pin digital** y solo lo encendemos al medir.

Montaje: sensor de humedad en A0, alimentado por el pin 7



Conexiones

Pin 7	→	VCC del módulo (col. 8)
Riel −	→	GND del módulo (col. 9)
A0	→	AO del módulo (col. 11)
GND	→	riel − de la protoboard

¿Por qué VCC va al pin 7 y no a 5V? Así el Arduino enciende el sensor solo cuando va a medir, y las patas duran mucho más. El pin DO no lo usamos.

Código: medir la humedad en porcentaje

HumedadTierra.ino

```ino
const int PIN_SENSOR = A0;
const int PIN_ENERGIA = 7;   // alimenta el sensor
const int SECO = 1000;       // valor en el aire (calibrar)
const int MOJADO = 400;      // valor en agua (calibrar)

void setup() {
  pinMode(PIN_ENERGIA, OUTPUT);
  digitalWrite(PIN_ENERGIA, LOW);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(PIN_ENERGIA, HIGH);   // encender sensor
  delay(10);
  int valor = analogRead(PIN_SENSOR);
  digitalWrite(PIN_ENERGIA, LOW);    // apagar sensor

  int humedad = map(valor, SECO, MOJADO, 0, 100);
  humedad = constrain(humedad, 0, 100);  // entre 0 y 100

  Serial.print("Valor: ");    Serial.print(valor);
  Serial.print("  Humedad: "); Serial.print(humedad);
  Serial.println(" %");
  if (humedad < 30) Serial.println("¡Tengo sed! Riégame");
  delay(1000);
}
```

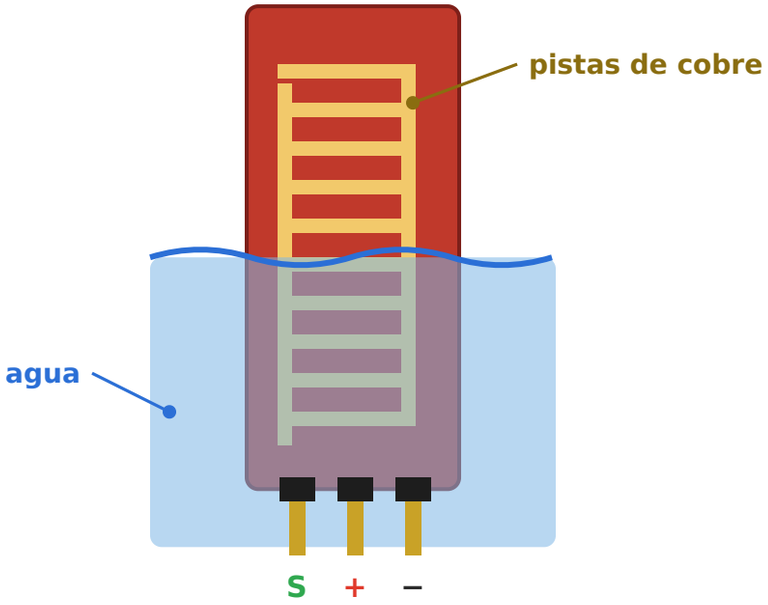
📏 Cómo calibrar

1. Sonda **al aire** (seca): anota el valor del monitor → **SECO**.
2. Sonda en un **vaso con agua** (sin mojar el módulo): anota el valor → **MOJADO**.
3. Escribe esos dos números en el código y súbelo otra vez.

constrain() mantiene el número entre 0 y 100, aunque la lectura se pase un poquito.

Fíjate: en `map()` pusimos SECO primero y MOJADO después. Así, más agua = porcentaje más alto.

Sensor de nivel de agua



¿Cómo funciona?

Tiene **pistas de cobre** intercaladas, como los dientes de dos peinillas. El agua las va uniendo: mientras **más hundido** esté el sensor, más pistas se conectan y **más alto** es el número.

Los números

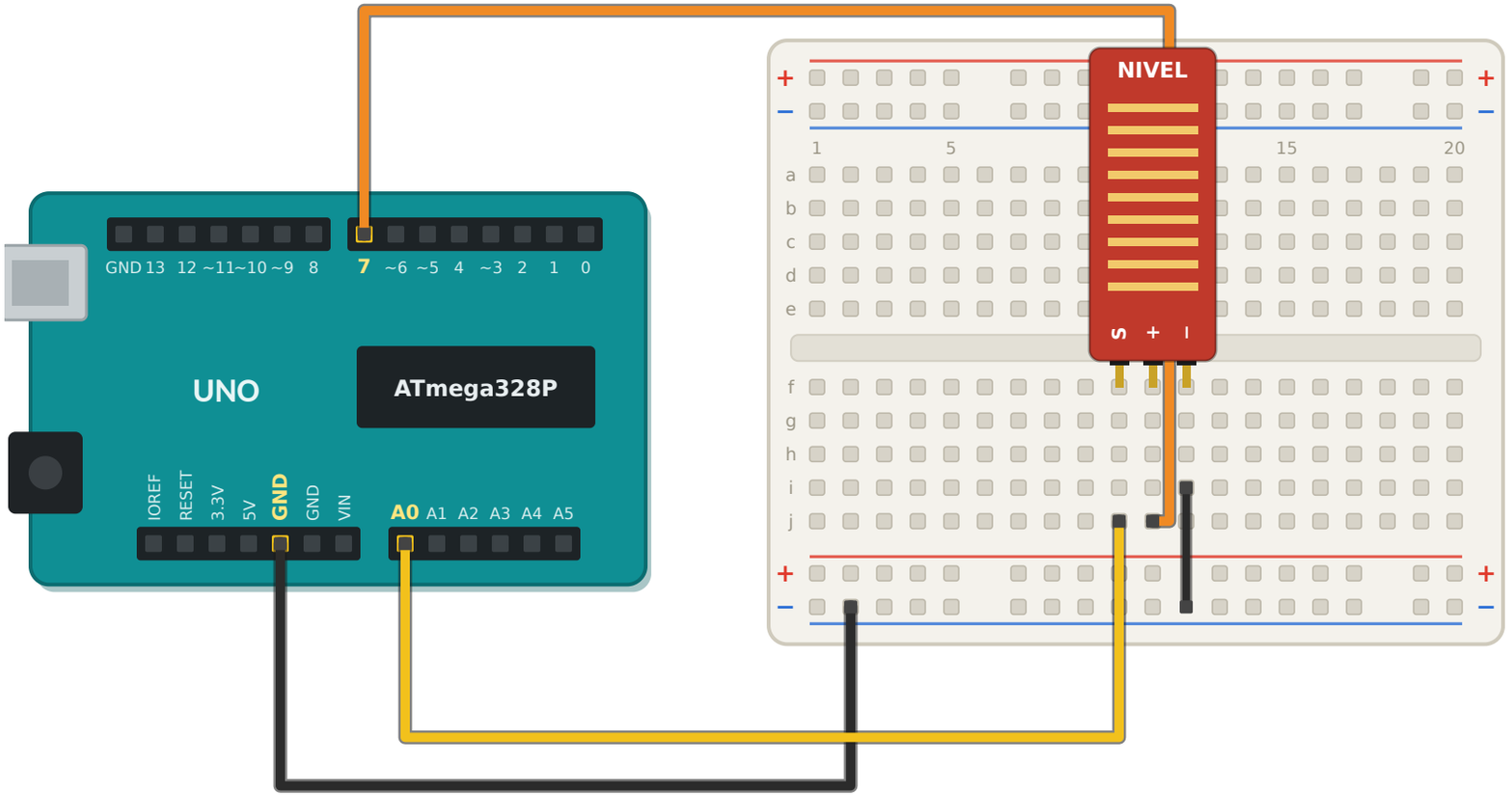
Seco → **0** (o casi).
Medio hundido → unos **400**.
Bien hundido → **500 a 700**, según el sensor y el agua.

⚠️ Cuidado

Solo se mojan las **pistas**, nunca los pines ni la parte de arriba. Igual que el de tierra, se oxida: lo alimentamos desde un **pin digital**.

Usos: alarma de inundación, avisar cuando la pecera o el tanque se están quedando sin agua.

Montaje: sensor de nivel en A0, alimentado por el pin 7



Conexiones

A0	→	S del sensor (col. 10)
Pin 7	→	+ del sensor (col. 11)
Riel −	→	− del sensor (col. 12)
GND	→	riel − de la protoboard

Para probarlo, usa un vaso con agua y hunde el sensor **poco a poco**, sin mojar los pines.

Código: ¿cuánta agua hay?

NivelAgua.ino

```ino
const int PIN_SENSOR = A0;
const int PIN_ENERGIA = 7;
const int ALERTA = 400;       // ajústalo a tu sensor

void setup() {
  pinMode(PIN_ENERGIA, OUTPUT);
  digitalWrite(PIN_ENERGIA, LOW);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(PIN_ENERGIA, HIGH);   // encender
  delay(10);
  int nivel = analogRead(PIN_SENSOR);
  digitalWrite(PIN_ENERGIA, LOW);    // apagar

  Serial.print("Nivel: ");
  Serial.println(nivel);

  if (nivel > ALERTA) {
    Serial.println("¡Cuidado! El agua está alta");
  }
  delay(1000);
}
```

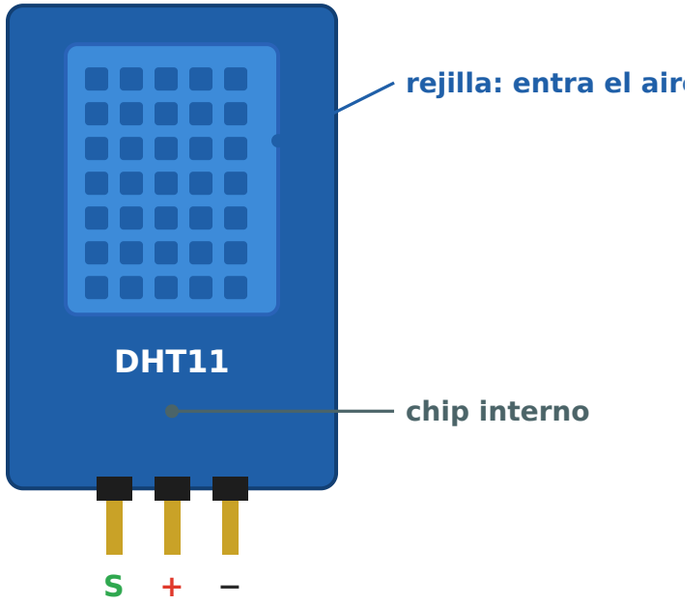
Es la **misma receta** que el sensor de tierra: encender → esperar un poquito → leer → apagar.

🧪 Experimenta

Hunde el sensor en 3 niveles (poco, medio y casi todo) y anota los valores. Con eso decide tu número de **ALERTA**.

¿Y si en vez de agua del grifo usas agua con sal? Pruébalo: la sal hace que el agua conduzca **mejor**. ¿Qué crees que pasará con el número?

DHT11: temperatura y humedad del aire



🌡️ Temperatura

De **0 a 50 °C**.
Precisión: ± 2 °C.

💧 Humedad del aire

De **20 a 90 %**.
Precisión: ± 5 %.

Un sensor “que habla”

Tiene un **chip propio**: mide y le envía al Arduino un **mensaje digital** con los dos números por un solo cable (pin S). Por eso no usamos analogRead(), sino una **librería** que entiende ese mensaje.

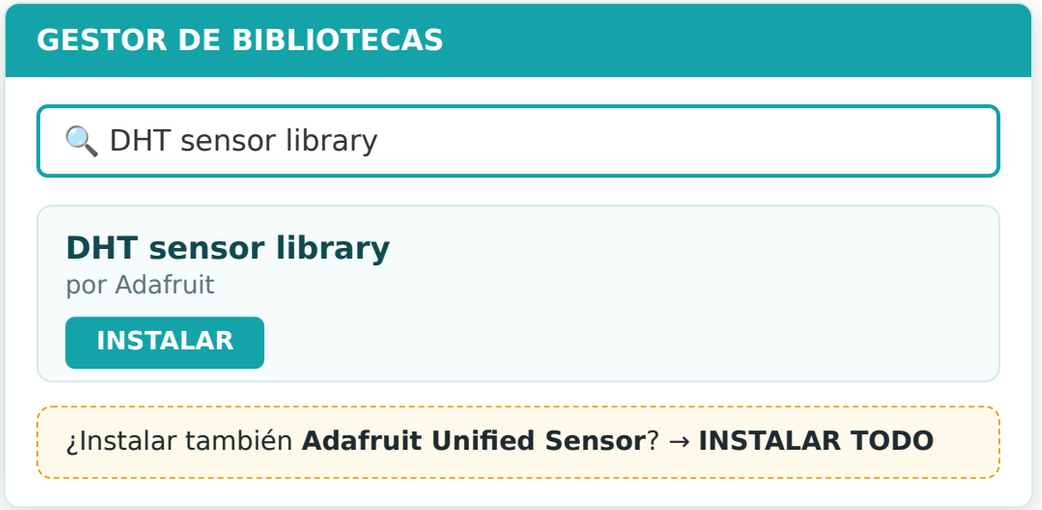
Es lento: solo da una lectura nueva cada 1 a 2 segundos. Si le preguntas más rápido, te repite el dato viejo o da error.

El orden de los pines cambia según el módulo (“S + −” o “+ OUT −”). Lee las letras de tu módulo.

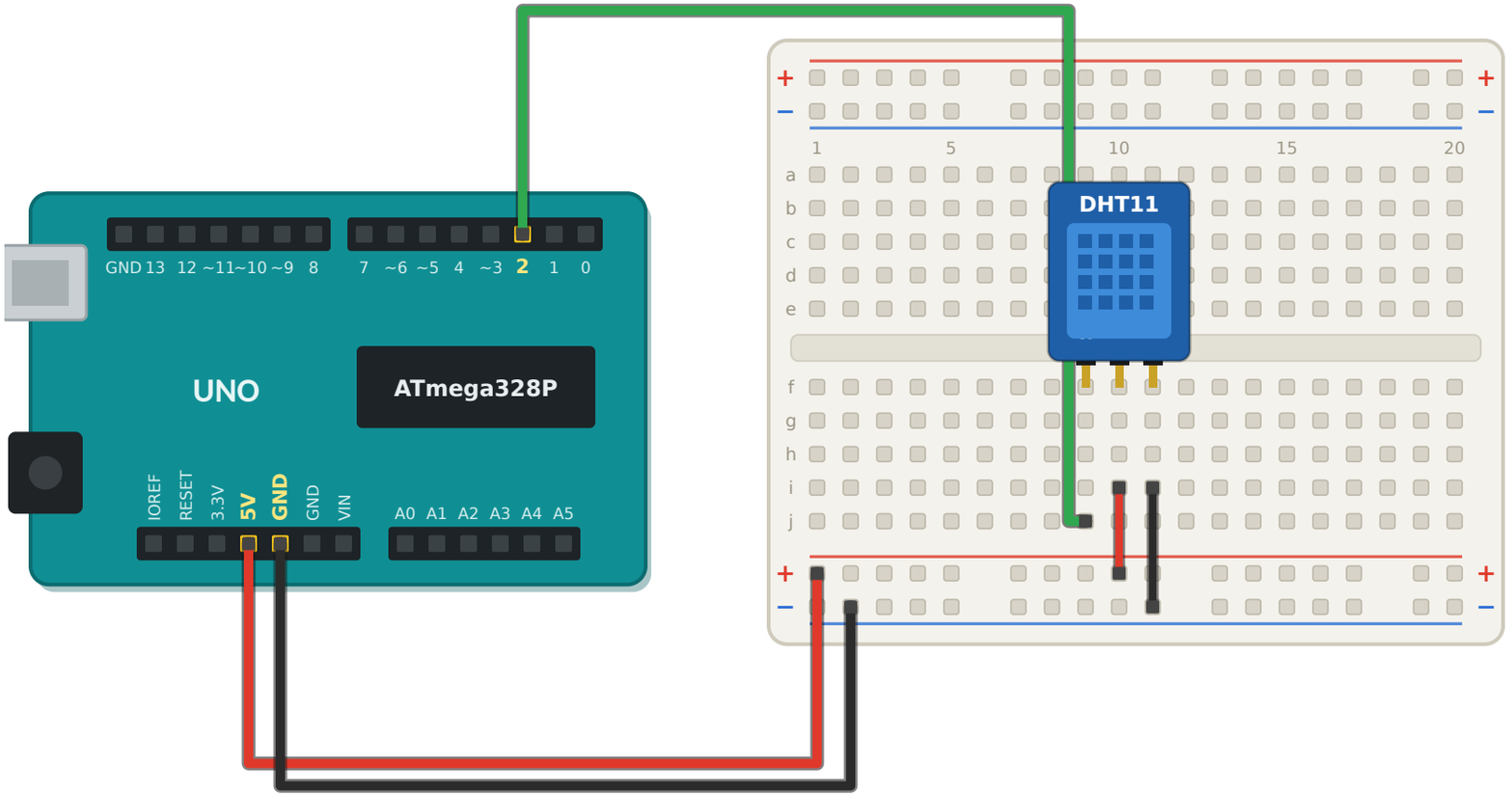
Antes de programar: instalar la librería del DHT

1. **Abre el gestor**
   En el Arduino IDE, haz clic en el ícono de **libros** de la barra izquierda (o **Ctrl + Shift + I**).
2. **Busca**
   Escribe **DHT sensor library**.
3. **Instala**
   Elige la de **Adafruit** y presiona **INSTALAR**.
4. **Acepta lo que necesita**
   Cuando pregunte por **Adafruit Unified Sensor**, elige **INSTALAR TODO**.

¿Qué es una librería? Código que otra persona ya escribió y comparte para que nosotros lo usemos. Se instala **una sola vez** en cada computador. Necesitas internet.



Montaje: DHT11 en el pin 2



Conexiones

Pin 2	→	S (datos) del sensor (col. 9)
Riel +	→	+ del sensor (col. 10)
Riel −	→	− del sensor (col. 11)
5V / GND	→	a los rieles + y −

El módulo ya trae la resistencia que necesita el DHT11. Si tu sensor fuera la versión **suelta de 4 patas**, habría que agregar una de 10 kΩ.

Código: tu propia estación del clima

EstacionClima.ino

```ino
#include <DHT.h>                  // la librería que instalamos

const int PIN_DHT = 2;
DHT dht(PIN_DHT, DHT11);          // creamos el sensor

void setup() {
  Serial.begin(9600);
  dht.begin();                    // lo encendemos
}

void loop() {
  delay(2000);                    // el DHT11 es lento
  float humedad = dht.readHumidity();
  float temperatura = dht.readTemperature();

  if (isnan(humedad) || isnan(temperatura)) {
    Serial.println("Error: revisa las conexiones");
    return;                       // vuelve a empezar el loop
  }
  Serial.print("Temperatura: ");
  Serial.print(temperatura);
  Serial.print(" °C   Humedad: ");
  Serial.print(humedad);
  Serial.println(" %");
}
```

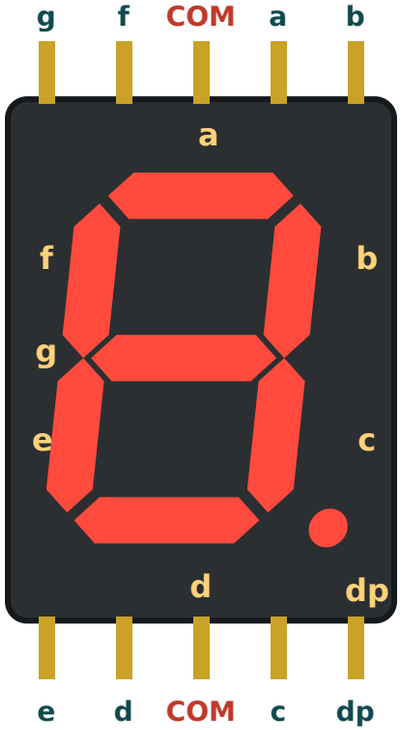
Monitor Serie

```
Temperatura: 28.00 °C   Humedad: 71.00 %
Temperatura: 28.00 °C   Humedad: 72.00 %
Temperatura: 29.00 °C   Humedad: 76.00 %
```

`isnan()` pregunta “¿no es un número?”. Si el sensor falló, mostramos un aviso en lugar de números raros.

🧪 **Experimenta:** sopla suavemente sobre el sensor. ¿Qué sube más rápido, la temperatura o la humedad?

El display de 7 segmentos: 8 LEDs en uno



pines de arriba

pines de abajo

¿Qué es?

Son **8 LEDs** con forma de rayita: 7 segmentos (de la **a** a la **g**) y un punto (**dp**). Encendiendo algunos y otros no, formamos los números del 0 al 9. Por eso lo llamamos “de 8 segmentos” o “de 7 segmentos con punto”.

Sus 10 pines

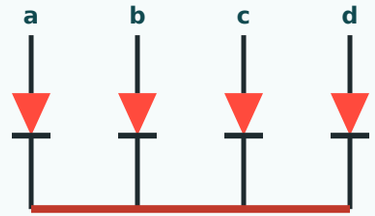
8 pines, uno para cada segmento, y 2 pines **COM** (común) en el medio, que están unidos por dentro. Basta con conectar uno de ellos.

Cada segmento es un LED

Por eso necesita **resistencia**. Lo ideal es una de 220 Ω por segmento. Para simplificar, hoy usamos **una sola en el pin COM** (los números con más segmentos se verán un poco más oscuros).

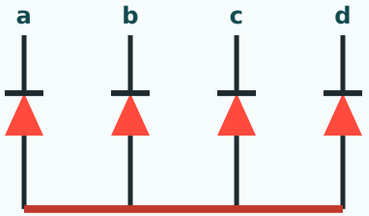
Mira el display con el número 8 encendido: así identificas cada letra. La “a” siempre es la de arriba y se sigue en el sentido del reloj.

¿Cátodo común o ánodo común? Descúbrelo



Cátodo común

segmento se enciende con HIGH



Ánodo común

segmento se enciende con LOW

En el código hay una línea para eso:

```
const bool ANODO_COMUN = false;
```

Si tu display resulta de ánodo común, cámbiala a **true**. ¡Nada más!

🔬 La prueba de los 30 segundos

1. Conecta un pin **COM** (el del medio) a **GND**.
2. Con una resistencia de 220 Ω, lleva **5V** a cualquier otra pata.
3. ¿Se encendió un segmento? → Tu display es de **cátodo común**.
4. ¿No encendió? Al revés: COM a **5V** y la resistencia a **GND**. Si ahora enciende → **ánodo común**.

Así también descubres qué pata enciende qué segmento, por si tu display tiene otro orden de pines.

La tabla secreta: qué segmentos forman cada número



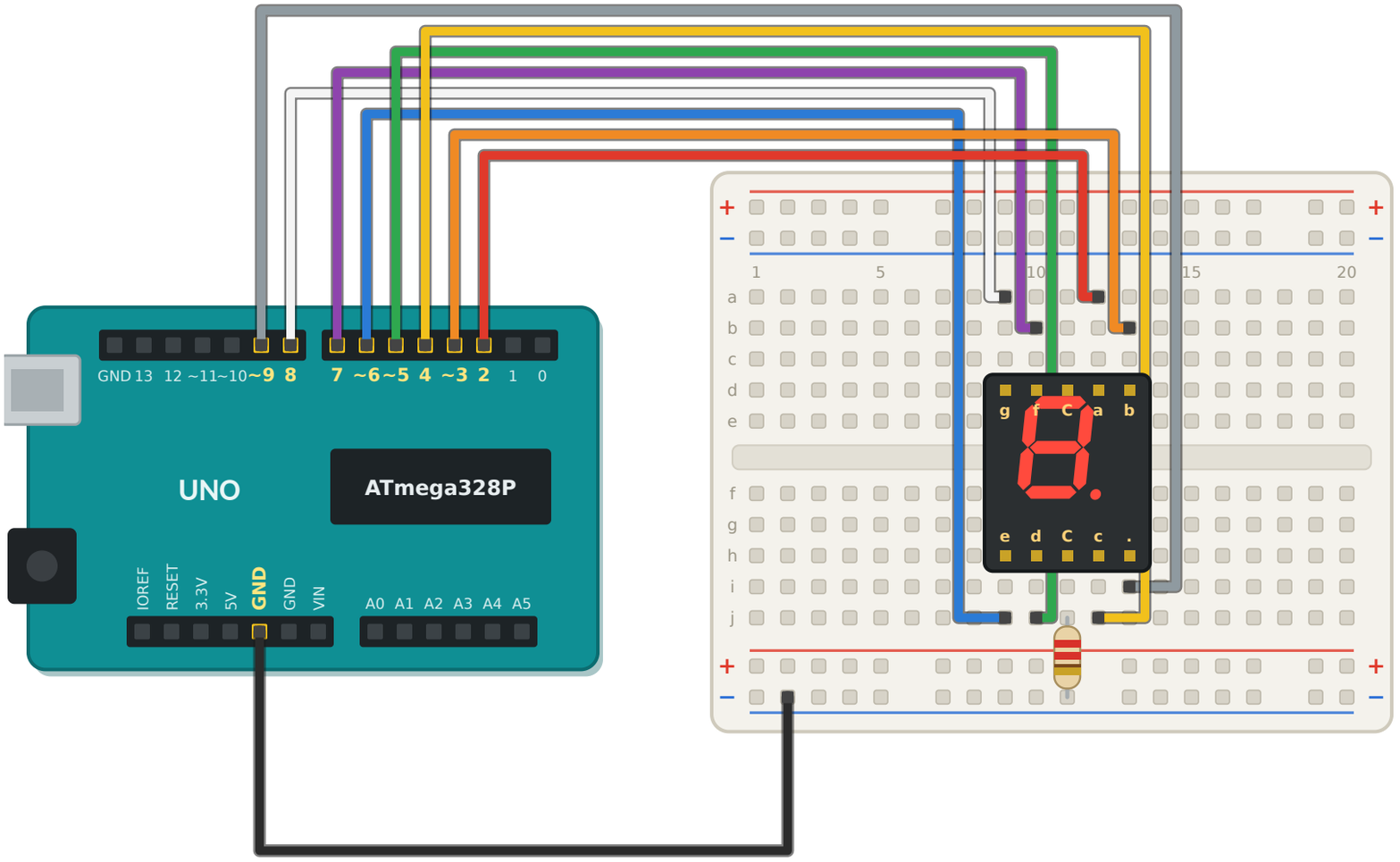
0 1 2 3 4 5 6 7 8 9

Para mostrar un **2** encendemos a, b, d, e y g. Esa lista de 1 y 0 la guardamos en el código como una **tabla** (un *arreglo*): una fila por cada número.

Reto relámpago: sin mirar la tabla, ¿qué segmentos necesitas para la letra **A**? ¿Y para la **H**?

Número	a	b	c	d	e	f	g
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2	1	1	0	1	1	0	1
3	1	1	1	1	0	0	1
4	0	1	1	0	0	1	1
5	1	0	1	1	0	1	1
6	1	0	1	1	1	1	1
7	1	1	1	0	0	0	0
8	1	1	1	1	1	1	1
9	1	1	1	1	0	1	1

Montaje: display en los pines 2 a 9



Conexiones (pin → segmento)

Pin 2 → a	→	col. 12, arriba
Pin 3 → b	→	col. 13, arriba
Pin 4 → c	→	col. 12, abajo
Pin 5 → d	→	col. 10, abajo
Pin 6 → e	→	col. 9, abajo
Pin 7 → f	→	col. 10, arriba
Pin 8 → g	→	col. 9, arriba
Pin 9 → dp	→	col. 13, abajo
COM (col. 11)	→	220 Ω al riel − (o a + si es ánodo)

Código: contar del 0 al 9

Display7Seg.ino

```
// pines de los segmentos:    a  b  c  d  e  f  g
const int SEGMENTOS[7] =    { 2, 3, 4, 5, 6, 7, 8 };
const bool ANODO_COMUN = false;   // true si es ánodo común
const int NUMEROS[10][7] = {      // 1 = encendido, 0 = apagado
  {1,1,1,1,1,1,0}, {0,1,1,0,0,0,0},   // 0 y 1
  {1,1,0,1,1,0,1}, {1,1,1,1,0,0,1},   // 2 y 3
  {0,1,1,0,0,1,1}, {1,0,1,1,0,1,1},   // 4 y 5
  {1,0,1,1,1,1,1}, {1,1,1,0,0,0,0},   // 6 y 7
  {1,1,1,1,1,1,1}, {1,1,1,1,0,1,1}    // 8 y 9
};
void setup() {
  for (int i = 0; i < 7; i++) pinMode(SEGMENTOS[i], OUTPUT);
}
void loop() {
  for (int n = 0; n <= 9; n++) {
    mostrar(n);
    delay(1000);
  }
}
void mostrar(int numero) {
  for (int i = 0; i < 7; i++) {
    int valor = NUMEROS[numero][i];
    if (ANODO_COMUN) valor = !valor;   // invierte si hace falta
    digitalWrite(SEGMENTOS[i], valor);
  }
}
```

📋 Arreglos (tablas)

`SEGMENTOS[7]` guarda 7 números en una sola variable. `SEGMENTOS[0]` es el primero (el 2), `SEGMENTOS[1]` el segundo… ¡Se cuenta desde 0!

🧩 Nuestra función

`mostrar(n)` recorre los 7 segmentos y enciende los que dicen 1 en la fila del número **n**. Como en la clase 2 con `luz()`.

⭐ Prueba

Cambia el loop para que cuente **hacia atrás**, del 9 al 0, como una cuenta regresiva.

El punto (pin 9) no lo usamos aquí. Puedes encenderlo con `pinMode(9, OUTPUT)` y `digitalWrite(9, HIGH)`.

RESUMEN

La chuleta de la clase

Componente	Tipo	Pin	Cómo se usa	Para recordar
Potenciómetro	Analógica	A0–A5	`analogRead()`	0 a 1023
Pulsador	Digital	2–13	`digitalRead() + INPUT_PULLUP`	LOW = presionado
Buzzer activo	Salida	2–13	`digitalWrite()`	HIGH = suena
Display 7 seg.	Salida	2–9	`digitalWrite() × 7`	tabla de números
Sensor IR	Digital	2–13	`digitalRead()`	LOW = obstáculo
Sonido	Analógica + digital	A0 y 2–13	`analogRead() / digitalRead()`	DO = HIGH con ruido
Humedad de tierra	Analógica	A0–A5	`analogRead()`	seca = número alto
Nivel de agua	Analógica	A0–A5	`analogRead()`	más agua = número alto
DHT11	Digital (datos)	2–13	`librería DHT`	cada 2 segundos

`Serial.begin(9600)` · `Serial.print()` · `Serial.println()` → ver datos

`map(valor, 0, 1023, 0, 255)` → cambiar de escala

`constrain(x, 0, 100)` → no salirse del rango

Retos nivel 1: para calentar

1. El timbre

🟢 **Nivel 1**

Mientras presionas el botón, el buzzer suena. Cuando lo sueltas, silencio.

Pulsador **Buzzer**

💡 **Pista:** usa el código del botón y cambia el LED por el buzzer.

2. Parpadeo a tu gusto

🟢 **Nivel 1**

El potenciómetro controla qué tan **rápido** parpadea un LED. Perilla a un lado: lento; al otro: rapidísimo.

Potenciómetro **LED**

💡 **Pista:** puedes usar el valor del potenciómetro directo dentro del `delay()`.

3. Medidor de giro

🟢 **Nivel 1**

Muestra en el Monitor Serie cuánto está girado el potenciómetro en porcentaje: *“Giro: 75 %”*.

Potenciómetro **Monitor Serie**

💡 **Pista:** `map(valor, 0, 1023, 0, 100)`.

4. Alarma de cercanía

🟢 **Nivel 1**

Cuando algo se acerca al sensor IR, el buzzer da 3 pitidos y el Monitor Serie dice *“¡Intruso!”*.

Sensor IR **Buzzer**

💡 **Pista:** junta el código del sensor IR con el del buzzer.

Hagan los retos en parejas: uno conecta y el otro programa. En el siguiente reto, cambian de rol.

Retos nivel 2: combinar

5. Sensor de reversa

🟡 Nivel 2

Como en los carros: el potenciómetro simula la distancia. Mientras más lo giras, el buzzer pita **más rápido**. Al tope, suena seguido.

Potenciómetro **Buzzer**

💡 **Pista:** `pausa = map(valor, 0, 1023, 1000, 50)` y úsala en los delay.

6. El semáforo de la planta

🟡 Nivel 2

Tres LEDs muestran la humedad de la tierra: **verde** (más de 60 %), **amarillo** (30 a 60 %) y **rojo** (menos de 30 %).

Humedad de tierra **3 LEDs**

💡 **Pista:** usa `if`, `else if` y `else`.

7. Contador de 0 a 9

🟡 Nivel 2

Cada vez que presionas el botón, el display suma 1. Después del 9 vuelve al 0.

Pulsador **Display**

💡 **Pista:** después de sumar, espera a que suelten el botón: `while (digitalRead(BOTON) == LOW) { }`.

8. Lámpara por aplauso

🟡 Nivel 2

Un aplauso enciende el LED; otro aplauso lo apaga.

Sensor de sonido **LED**

💡 **Pista:** guarda el estado en `bool encendido` y usa `encendido = !encendido;`.

Antes de programar, dibujen en el cuaderno cómo van a conectar todo.

Retos nivel 3: proyectos

9. Alarma de inundación

🔴 Nivel 3

Si el nivel de agua pasa tu número de alerta: el buzzer suena a ratos, un LED rojo parpadea y el Monitor Serie avisa. Si baja, todo se apaga.

Nivel de agua **Buzzer** **LED**

💡 **Pista:** primero mide cuánto marca tu sensor en un vaso, luego decide la alerta.

10. Termómetro con display

🔴 Nivel 3

Muestra la temperatura del DHT11 en el display, un dígito a la vez: primero las decenas y luego las unidades (2… 7…), con una pausa entre cada lectura.

DHT11 **Display**

💡 **Pista:** `decenas = temp / 10;` y `unidades = temp % 10;` (con `int temp`).

11. Dado electrónico

🔴 Nivel 3

Al presionar el botón, el display muestra números al azar muy rápido (como si girara) y se detiene en un número del 1 al 6.

Pulsador **Display** **Buzzer (opcional)**

💡 **Pista:** `random(1, 7)` da un número del 1 al 6. Usa un `for` para la animación.

12. Mi invento

🔴 Nivel 3

Inventa algo útil para tu casa o tu colegio que use **al menos 1 sensor y 2 salidas**. Ponle nombre y preséntalo a la clase.

¡Tú eliges!

💡 **Pista:** piensa primero: ¿qué problema resuelve? Luego: ¿qué sensor lo detecta? ¿Cómo avisa?

No hace falta hacerlos todos: elijan los que más les gusten y háganlos funcionar bien.

Errores comunes con sensores (y cómo salir de ellos)

El valor siempre es 0 o 1023

El cable de señal está suelto o en otro pin. Revisa que esté en **A0** y no en el pin digital 0.

Los números saltan solos

Pin “flotando”: falta un GND compartido, o el botón no tiene `INPUT_PULLUP`.

El monitor muestra símbolos raros

La velocidad del monitor no es igual a `Serial.begin(9600)`.

El monitor está vacío

¿Olvidaste `Serial.begin(9600)` en el setup()? ¿Elegiste el puerto correcto?

El sensor se calienta

¡Desconecta el USB ya! Seguramente **VCC y GND están al revés**.

El DHT11 da “Error”

Revisa el pin de datos, que la librería esté instalada y que esperes 2 segundos entre lecturas.

No sube el programa

Desconecta lo que tengas en los pines 0 y 1 y vuelve a intentarlo.

El display muestra cosas raras

Revisa el orden de los cables a–g y si tu display es de ánodo común.

El Monitor Serie es tu mejor amigo: si algo no funciona, imprime los valores y mira qué está pasando.

CIERRE · CLASE 3

Hoy el Arduino aprendió a escuchar

✅ Leemos resistencias por sus colores
✅ Vemos lo que piensa el Arduino con el Monitor Serie
✅ Entendemos el potenciómetro por dentro
✅ Usamos botón, buzzer, display y 5 sensores

📝 Para la próxima clase

Cada pareja elige **un reto de nivel 2 o 3** y lo trae funcionando.

Prepárense para explicar en 1 minuto: ¿qué hace?, ¿qué sensor usaron? y ¿qué fue lo más difícil?

Pregunta para pensar: si combinamos el sensor de humedad, una bomba de agua y el Arduino… ¿qué podríamos construir?